Detección de cambios de punto en series temporales para la prevención de averías en la línea de producción de Ford

Joan Camps Ciria
Universidad CEU Cardenal Herrera
camcirjoa@alumnos.uchceu.es

Julio 2019

"Sólo podemos ver un poco del futuro, pero lo suficiente para saber que hay mucho por hacer."

Alan Mathison Turing.(1912-1954)

Agradecimientos

A todos los profesores, por aguantarme durante todos estos años, aunque no nos vieramos mucho. Gracias por aprender tanto con vosotrosy ayudarme siempre que lo he necesitado

A mis padres, por su incondicional apoyo reiterado y costante.

Resumen

Este proyecto analiza en profundidad algorítmos de detección de cambios de punto en series temporales para la prevención de averías en las factorias de Ford. En el proyecto se analiza una serie de repositorios o paquetes de R, en los cuales encontramos una gran cantidad de algoritmos que estan programados para la detección de cambios de punto en series temporales o arrays. Se pretende analizar estos algorítmos con el fin de encontrar en una serie de tiempos una fructuación o cambio de punto que nos permita comprender que patología sufre una maquina/robot en una línea de producción. Gracias a los resultados de la tesis doctoral de Eduardo García Magraner [5], se propuso un nuevo modelo matemático que permitía el análisis de los sub-tiempos de ciclo técnico para la mejora del rendimiento de las lineas de fabricación. Este modelo subdivide los términos a corto plazo, tiempo que le cuesta a una máquina realizar una determinada tarea, en sub-tiempos, pudiendo determinar como se están deteriorando los distintos componentes. Con la medición de esos subtiempos, unido a la evaluación del resultado a través de los algoritmos de cambio de punto, se puede extraer conocimiento que nos permita predecir el deterioro de un componente en una máquina.

Índice general

1. Introducción 4
   1.1. Innovación Tecnológica y Empresarial 4
   1.2. La Cuarta Revolución Industrial 5
   1.3. Big Data 6
   1.4. Definición cadena de producción 7
   1.5. Diseño de la línea 7
2. Antecedentes 10
   2.1. Consideraciones previas 10
   2.2. Toma de datos 10
   2.3. Granularidad de los datos 10
   2.4. Modos de funcionamiento de una estación de trabajo 10
   2.5. Medición experimental de la estación de trabajo 11
   2.6. Patologías aplicables a las estaciones de soldadura 13
   2.7. Mediciones según patologías 14
3. Análisis de métodos de punto de cambio 16
   3.1. Repositorio Trend [6] 16
   3.2. Repositorio Strucchange [7] 25
   3.3. Repositorio wbsts [8] 34
   3.4. Repositorio wbs 36
   3.5. Repositorio cpm 40
   3.6. Repositorio not 50
4. Resultados de cada algoritmo de punto de cambio para cada patología 54
   4.1. Repositorio trend 54
5. Objetivo del proyecto 57
   5.1. Definición Cuadro de Mando 57
   5.2. KPI's para Ford Valencia 58
6. Aplicación Cuadro de Mando 59
   6.1. Concepto general 59
   6.2. Elección de lenguajes y tecnologías 59
   6.3. Captura de datos 60
   6.4. Funcionamiento de la aplicación 60
   6.5. Instalaciones en el servidor 61
   6.6. Instalaciones en los clientes 64
7. Conclusiones y trabajos futuros 80

Índice de figuras

1. *Sistema de automatización y control distribuido en una Industria 4.0. [3]* . . . 5
2. *Gráfica con datos de producción reales de la planta de carrocería de Ford Valencia. [5].* . . . 8
3. *Estación de soldadura [5].* . . . 11
4. *Funcionamiento básico de una estación de trabajo en Ford Valencia [5].* 11
5. *Medidas experimentales de los mini-términos para una estación de soldadura [5].* . . . 12
6. *Configuración Experimental de la estación de trabajo. [5].* . . . 13
7. *Medidas experimentales para cada mini-término y cada patología [5].* . 14
8. *Medidas experimentales de ciclo independiente para cada mini-término y cada patología [5].* . . . 15
9. *Plot del resultado del test de Pettitt.* . . . 17
10. *Plot del resultado del test de Buishand.* . . . 20
11. *Plot del resultado del test U de Buishand.* . . . 22
12. *Plot del resultado de la prueba estándar de homogeneidad normal.* . . 24
13. *Plot de los datos de ejemplo Nile.* . . . 26
14. *Plot de Fstats de Nile.* . . . 26
15. *Sctest de Nile* . . . 27
16. *Breakpoints de Nile* . . . 27
17. *Fstats de los datos de Ford* . . . 29
18. *Proceso OLS-CUSUM con los datos de Ford* . . . 30
19. *Proceso Rec-CUSUM con los datos de Ford* . . . 31
20. *Proceso OLS-MOSUM con los datos de Ford* . . . 32
21. *Proceso Rec-MOSUM con los datos de Ford* . . . 33
22. *Plot Binary Segmentation* . . . 37
23. *Plot Binary Segmentation* . . . 39
24. *Estado de la simulación de las máquinas en una estación. [5].* . . . 56
25. *Pantalla principal del IDE Qt Creator.* . . . 60
26. *Cuota de mercado de sistemas operativos en España (%) [21].* . . . 61
27. *Estructura de la base de datos en el servidor.* . . . 63
28. *Esquema del proceso completo (Fuente propia).* . . . 64
29. *Aplicación instalada en un smartphone.* . . . 65
30. *Pantalla de login en smartphone.* . . . 65
31. *Pantalla para usuarios de gerencia en smartphone (Tiempos TcT).* . . 66
32. *Pantalla para usuarios de gerencia en smartphone (detalle cuello de botella).* . . . 66

33. *Pantalla para usuarios de gerencia en smartphone (Producción mensual).* 67
34. *Pantalla para usuarios de mantenimiento en smartphone.* 67
35. *Pantalla de login en tablet.* 68
36. *Pantalla para usuarios de gerencia en tablet (Tiempos TcT).* 68
37. *Pantalla para usuarios de gerencia en tablet (detalle cuello de botella).* 69
38. *Pantalla para usuarios de gerencia en tablet (Producción mensual).* . 69
39. *Pantalla para usuarios de mantenimiento en tablet.* 70
40. *Pantalla principal del instalador de la aplicación.* 70
41. *Pantalla para seleccionar ubicación de la aplicación.* 71
42. *Pantalla de confirmación.* 71
43. *Pantalla con el progreso de la instalación.* 72
44. *Pantalla con el resultado de la instalación.* 72
45. *Acceso directo de la aplicación en el escritorio.* 72
46. *Pantalla de introducción de usuario y contraseña.* 73
47. *Pestaña de Tiempos.* 74
48. *Pantalla de detalle del cuello de botella (CdB).* 75
49. *Pestaña de Producción.* 76
50. *Pestaña de Valores TcT/Offset.* 77
51. *Pestaña de Idle Time (Tiempo Ocioso).* 78
52. *Pantalla con el detalle de los tiempos de actividad de cada minitérmino monitorizado.* 78
53. *Pantalla con la alerta de superación del umbral máximo.* 79

1. Introducción

1.1. Innovación Tecnológica y Empresarial

Poco a poco la tecnología deja de ser un privilegio para todo tipo de personas. A día de hoy se ha transformado en una pieza clave dentro del área empresarial y personal [1].

Es practicamente inexplicable todo el avance que se ha producido en el ámbito empresarial gracias a la tecnología, fácilmente, en las situaciones que, anteriormente, llevaban semanas o meses llevarlas a cabo, hoy en día culminan en minutos o unas pocas horas. También en el esfuerzo; ya no se necesita tanto como décadas pasadas.

Las nuevas tecnologias en las empresas se han vuelto esenciales, y cuando estas no optan por implementarlas en los diferentes procesos, las probabilidades de estancarse y perder posicionamiento aumentan considerablemente y más si se encuentran en pleno crecimiento.

- Mayor productividad. Cuando una empresa esta tecnologicamente actualizada se reducen las horas de trabajo manual y aumenta la productividad a escalas increíbles. Obviamente, dichas tecnologías deben estar enfocadas en el objetivo de cumplir aquellas necesidades que tenga el negocio y, en paralelo acompañar a las metas del mismo.

- Procesos óptimos. Con estas tecnologías se pueden obtener mejoras en los procesos, una optimización fundamental, organización, cobranzas y ventas, despacho, etcétera. Se logra obtener ventajas competitivas para alcanzar un posicionamiento adecuado dentro del mercado.

- No a las barreras en la comunicación. Con las nuevas tecnologias en las empresas la comunicación entre ejecutivos, empleados, proveedores y clientes avanzará más que en una simple llamada telefónica, ya que se puede recurrir a diferentes maneras de ejercer susodicha comunicación, como, video llamadas, boletines, redes sociales, etcétera.

- Oficinas en cualquier lugar. Gracias a la tecnología el trabajo no se limitará a una estructura de cinco paredes, se podrá responder a los asuntos desde cualquier lugar. También se puede compartir información e interactuar con la red de contactos.

- Toma de decisiones mejorada. Los factores tecnologicos de una empresa implican el poder proveer información operativa adaptada al tiempo real, de tal forma que, en todo momento, se encuentre disponible para ser consumida por

directores y/o gerentes que tiene la empresa. Con la influencia del Big Data, la inteligencia de los sistemas está ayudando a las máquinas a dar soporte a la toma de decisiones.

1.2. La Cuarta Revolución Industrial

La industria de manufactura está experimentando una rápida evolución (o revolución) hacia lo que algunos han denominado Industria 4.0. En este nuevo paradigma las factorías están altamente automatizadas e informatizadas, todos sus procesos se encuentran conectados e interactúan entre sí y con procesos externos. Con sistemas de instrumentación, PLC's (Programmable Logic Controllers), CNCs (Computer Numerical Control), se generan grandes cantidades de datos que deben ser manipulados para convertirlos en información útil en cada uno de los diversos niveles del sistema de fabricación: máquina, celda de fabricación, línea de montaje, oficina técnica, gestión de producción, etc.

En la Figura 1 se puede ver un sistema de automatización y control distribuido en una Industria 4.0. Diversos controladores conectados entre sí mediante una red de tiempo real, y a la vez conectados con una segunda red a aplicaciones locales y/o remotas.

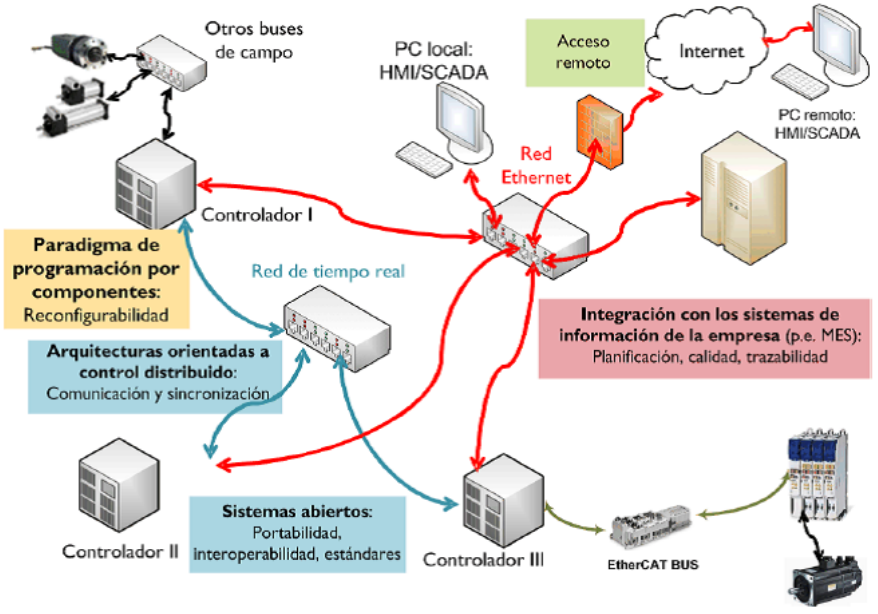


Figura 1: *Sistema de automatización y control distribuido en una Industria 4.0. [3]*

Actualmente existen tecnologías consolidadas en la industria como son los PLC's, tecnologías que han estado conviviendo con la industria durante mucho tiempo, pero que hoy en día no se adaptan a los cambios que esta demanda. Los nuevos paradigmas surgidos en la industria como el *Agile Manufacturing* introducen conceptos como la reconfiguración dinámica de los sistemas de fabricación, y en particular del software. Aunque lo más importante se centra en la conectividad en la Industria 4.0.

En este aspecto surgen nuevas tecnologías aplicables como el *Big Data*, el *Cloud Manufacturing* y la *Internet of Things (IoT)* que vienen a introducir nuevos estándares en la manera de producir [3].

1.3. Big Data

Cuando hablamos de Big Data nos referimos a conjuntos de datos o combinaciones de conjuntos de datos cuyo tamaño (volumen), complejidad (variabilidad) y velocidad de crecimiento (velocidad) dificultan su captura, gestión, procesamiento o análisis mediante tecnologías y herramientas convencionales, tales como bases de datos relacionales y estadísticas convencionales o paquetes de visualización, dentro del tiempo necesario para que sean útiles.

La naturaleza compleja del Big Data se debe principalmente a la naturaleza no estructurada de gran parte de los datos generados por las tecnologías modernas, como los web logs, la identificación por radiofrecuencia (RFID), los sensores incorporados en dispositivos, la maquinaria, los vehículos, las búsquedas en Internet, las redes sociales como Facebook, computadoras portátiles, teléfonos inteligentes y otros teléfonos móviles, dispositivos GPS y registros de centros de llamadas.

En la mayoría de los casos, con el fin de utilizar eficazmente el Big Data, debe combinarse con datos estructurados (normalmente de una base de datos relacional) de una aplicación comercial más convencional, como un ERP (Enterprise Resource Planning) o un CRM (Customer Relationship Management).

Casos de uso de Big Data [4]

El big data puede ayudarle a abordar una serie de actividades empresariales, desde la experiencia de cliente hasta la analítica:

- Desarrollo de productos. Empresas como Netflix y Procter & Gamble usan big data para prever la demanda de los clientes. Construyen modelos predictivos para nuevos productos y servicios clasificando atributos clave de productos anteriores y actuales, y modelando la relación entre dichos atributos y el éxito comercial de las ofertas.

- Mantenimiento predictivo. Los factores capaces de predecir fallos mecánicos pueden estar profundamente ocultos entre datos estructurados (año del equipo, marca o modelo de una máquina) o entre datos no estructurados que cubren millones de entradas de registros, datos de sensores, mensajes de error y temperaturas de motor. Al analizar estos indicadores de problemas potenciales antes de que estos se produzcan, las organizaciones pueden implantar el mantenimiento de una forma más rentable y optimizar el tiempo de servicio de componentes y equipos.

- Experiencia del cliente. Disponer de una vista clara de la experiencia del cliente es más posible que nunca. El big data permite recopilar datos de redes sociales, visitas a páginas web, registros de llamadas y otras fuentes de datos para mejorar la experiencia de interacción, así como maximizar el valor ofrecido.

- Fraude y conformidad. En lo que a seguridad se refiere, la empresa no se enfrenta a simples piratas informáticos deshonestos, sino a equipos completos de expertos. Los contextos de seguridad y requisitos de conformidad están en constante evolución. El big data ayuda a identificar patrones en los datos que pueden ser indicativos de fraude, al tiempo que concentra grandes volúmenes de información para agilizar la generación de informes normativos.

- Aprendizaje automático. El aprendizaje automático es actualmente un tema de gran actualidad. Los datos —concretamente big data— son uno de los motivos de que así sea. Ahora, en lugar de programarse, las máquinas pueden aprender. Esto es posible gracias a la disponibilidad de big data para crear modelos de aprendizaje automático.

- Eficiencia operativa. Es el área en que big data tiene un mayor impacto. El big data permite analizar y evaluar la producción, la opinión de los clientes, las devoluciones y otros factores para reducir las situaciones de falta de stock y anticipar la demanda futura. El big data también puede utilizarse para mejorar la toma de decisiones en función de la demanda de mercado en cada momento.

1.4. Definición cadena de producción

Según Wikipedia [13], *"la producción en cadena, producción en masa, producción en serie o fabricación en serie fue un proceso revolucionario en la producción industrial cuya base es la cadena de montaje o línea de ensamblado o línea de producción; una forma de organización de la producción que delega a cada trabajador una función específica y especializada en máquinas también más desarrolladas"*. Esta idea teórica sobre las cadenas de montaje surge con el taylorismo y el primero que tuvo la idea de ponerla en práctica, fue Ransom Olds, quien inauguró su cadena de montaje en 1901 construyendo su prototipo denominado Curved Dash. Sin embargo, el sistema de cadena de montaje tomó popularidad unos años después, gracias a Henry Ford, quien tomando la idea de Olds, desarrolló una cadena de montaje con una capacidad de producción superior y de la cual su producto emblemático fue el Ford T.

1.5. Diseño de la línea

El diseño de una línea o cadena de montaje no es algo trivial, supone un enorme esfuerzo de ingeniería para llegar a concretar un modelo que suponga un verdadero

ahorro en costes y lo más importante un aumento de la producción, con el consiguiente aumento de los beneficios. Todo este proceso se conoce como ALB (Assembly Line Balancing, por sus siglas en ingles), y comprendería todos los diseños y planificaciones de las líneas de producción. De todas formas no es fácil llegar a un ALB correcto para el problema en cuestión, de hecho, existe una brecha entre los requerimientos de problemas de configuraciones reales y el estado de las investigaciones sobre el tema.

Según [14], una de las razones de este déficit podría ser el hecho de que los trabajos de investigación a menudo consideran extensiones individuales o sólo unos pocos ALB en forma aislada. Es decir, no existen algoritmos adaptables a los diferentes problemas a los que se enfrenta la industria, cada problema es diferente y las soluciones propuestas no son lo suficientemente flexibles como para solventarlos. Esta brecha entre investigación y requerimientos va en aumento.

Hemos explicado que para el diseño de una línea de producción, en este caso de soldadura hemos tenido que recurrir a un proceso de balanceo de línea de ensamblaje (ALB), pero, ¿qué pasa después de este proceso?, lo ideal es que obtuviéramos unas tasas de producción máxima definidas en la fase de diseño y calculadas experimentalmente, en este caso medidas en JPH (Jobs per Hour). A este valor se le conoce como ERC (Engineering Running Capacity) y es trabajo de los operarios de la planta llegar a él. Sin embargo vemos que ese valor es demasiado optimista y por mucho que lo intentemos nunca obtenemos mediciones de JPH cercanas al valor propuesto. ¿Qué ha ocurrido?, ¿Algo se ha diseñado mal en la línea?. En la figura 2 se muestra una gráfica con los datos de producción reales de la planta de carrocería de Ford Valencia.

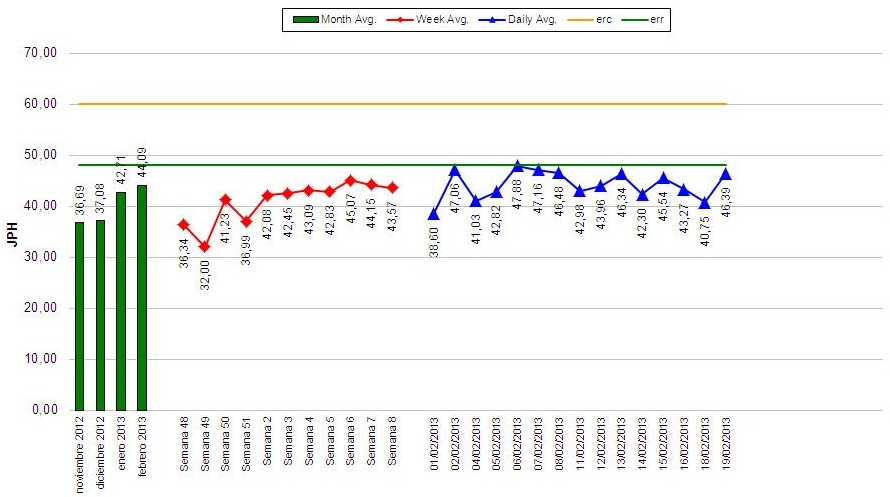


Figura 2: *Gráfica con datos de producción reales de la planta de carrocería de Ford Valencia. [5].*

Viendo la figura anterior nos damos cuenta de que ese valor teórico definido en

el diseño de la línea es inalcanzable, debido a diversos factores que muchas veces son imposible de parametrizar y controlar para insertarlos en un algoritmo de diseño. Entre los diversos factores causantes de la diferencia entre las dos mediciones podemos destacar las tareas de mantenimiento, las averías de los equipos, el deterioro de los componentes, etc. Con esta información se define un nuevo valor de medida denominado ERR (Engineering Running Rate) que supone una medición más realista de la producción.

2. Antecedentes

2.1. Consideraciones previas

2.2. Toma de datos

2.3. Granularidad de los datos

En [5], se pone de manifiesto que anteriormente a la tesis el nivel máximo de granularidad o análisis era como mucho en términos a corto plazo [11], es decir, períodos de funcionamiento de una máquina en concreto (unidad de soldadura). En la tesis se redefine este concepto y se crean dos nuevos que dividen al anterior en una nueva clasificación de granularidad: el mini-termino y el micro-termino.

El mini-termino se define como una parte de la máquina en una política de mantenimiento preventivo o en una avería en la que podría ser sustituida de una manera mas fácil y rápida que otra, o dicho de otro modo, una subdivisión de la máquina que nos permite comprender y estudiar el comportamiento de la misma. Del mismo modo, un micro-término se define como cada una de las partes, con interés de estudio o seguimiento, en las que puede ser dividido un mini-término.

Se redefine también para este caso el concepto de tiempo de ciclo probabilístico, ya que se tienen en cuenta estas subdivisiones de procesos. Por tanto, el tiempo de ciclo probabilístico de cada máquina es una concatenación de los tiempos de ciclo probabilísticos de los mini-términos y micro-términos donde cada uno puede tener un modelo de distribución probabilística diferente.

2.4. Modos de funcionamiento de una estación de trabajo

En la Figura 3 se muestra la estación de soldadura objeto de estudio. Esta estación es la que se cogió como muestra (todas las de la línea son iguales) para medir los tiempos de ejecución de cada proceso y evaluar su comportamiento. Esta estación se divide en tres partes, que coinciden con los tres mini-términos definidos. Por un lado tenemos el mini-término del brazo robot, después tenemos el mini-término del movimiento de soldadura y por último el mini-término de la propia acción de soldar.

La estación de trabajo o soldadura se comporta de la siguiente manera, primero el brazo robot mueve la pinza hasta el punto a soldar, después el cilindro neumático del robot mueve la pinza de soldadura en dos pasos, primero la aproxima al punto de soldadura y después realiza la acción de soldar.

El modo de funcionamiento básico de una estación de trabajo en una línea de soldadura de Ford Valencia es el mostrado en la Figura 4, cuya función se describe a continuación:

Hay tres posibles estados en las estaciones de trabajo: [trabajo, espera y bloqueo]. Si la estación actual está en estado de [trabajo] y el trabajo está terminado,

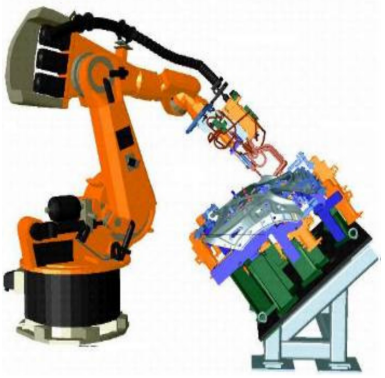


Figura 3: *Estación de soldadura [5].*

comprueba la siguiente estación; si ésta se encuentra en [espera] de pieza, la pieza acabada se entrega a la misma y el estado de la estación actual pasaría a estar libre de recibir otro trabajo. Si la siguiente estación se encuentra en estado de [trabajo] cuando la actual termina su labor, la estación actual cambia su estado a [bloqueo], es decir, el bloqueo de sí misma hasta que la siguiente estación este libre. Si la estación actual está libre para recibir otra pieza, comprueba la estación anterior, si la estación anterior se encuentra en estado de [trabajo], el actual estado cambia a [espera], es decir, permanece en espera hasta que la estación anterior tiene piezas para trabajar. Si la estación anterior se encuentra en estado de [bloqueo], la actual estación recibe la pieza y cambiará su estado a [trabajo]. Siempre hay una pieza disponible para trabajar en la primera estación y siempre se puede extraer la pieza terminada de la última estación. Cuando se inicia la simulación, todos los estados de la estación están establecidos en [espera] de pieza, pero la primera estación se establece en estado de [trabajo].

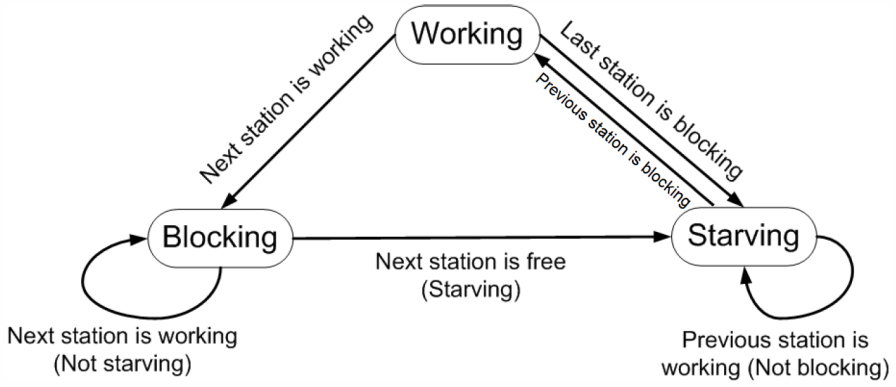


Figura 4: *Funcionamiento básico de una estación de trabajo en Ford Valencia [5].*

2.5. Medición experimental de la estación de trabajo

Para poder medir los tiempos de proceso de cada mini-término, se aisló una unidad de soldadura, lo que permitió obtener una base sobre la que poder trabajar

para posteriormente compararla con el proceso medido en tiempo real, calcular las desviaciones y poder hacerles frente.

Cada acción de soldar se repitió 6 veces por cada muestra obtenida. La trayectoria del robot fue la misma para cada prueba. Se realizaron 40 acciones de sujeción, para poder tener un abanico suficiente de datos para posteriormente calcular la media y la desviación típica de cada mini-término. La Figura 5 muestra las mediciones obtenidas para cada mini-término $(\bar{x}, S)$.

La estación escogida para la simulación, se corresponde con una estación de soldadura de carrocerías, una de las mas importantes de la planta, ya que de media se realizan alrededor de 4.500 puntos de soldadura para un modelo dado.

La estación de soldadura se compone básicamente de un brazo robot y una pinza de soldadura. La tarea de soldar de la estación se realiza en dos fases, primero el brazo robot mueve la pinza hasta el punto a soldar; después un cilindro neumático mueve la pinza de soldadura en dos fases, la primera, que aproxima la pinza a la pieza y la segunda que efectúa la acción de soldar. Con el fin de analizar la forma en que la estación realiza su trabajo, ésta se divide en tres partes denominadas mini-términos (brazo robot, movimiento de soldadura y acción de soldar).

La Figura 6 muestra el esquema para la obtención de las mediciones sobre la estación de soldadura. La prueba consiste en que el brazo robot inicialmente mueve la pinza hasta el punto a soldar, posteriormente el cilindro neumático aproxima la pinza hasta las piezas a soldar y les aplica presión hasta alcanzar el optimo, finalmente la pinza aplica corriente a los electrodos que sueldan las piezas.

	Mini-términos		
	Brazo robot (μ, σ)	**Pinza de soldadura (μ, σ)**	
	Posicionamiento	**Movimiento**	**Soldadura**
Medida total	(35.55, 0.67)	(2.49, 1.15)	(8.62,2.05)
Ciclo independiente	(1, 0.11)	(0.42, 0.47)	(1.44, 0.84)

Figura 5: *Medidas experimentales de los mini-términos para una estación de soldadura [5].*

Para las medidas del ciclo independiente, es decir, un solo tiempo de ciclo, se han utilizado las siguientes fórmulas matemáticas:

$$\mu = \frac{\mu}{6}$$
$$\sigma = \sqrt{\frac{\sigma^2}{6}}$$

En el caso de un brazo robot, el tiempo base es el tiempo de ciclo normal, que nos da el tiempo de ciclo por segundo, es decir:

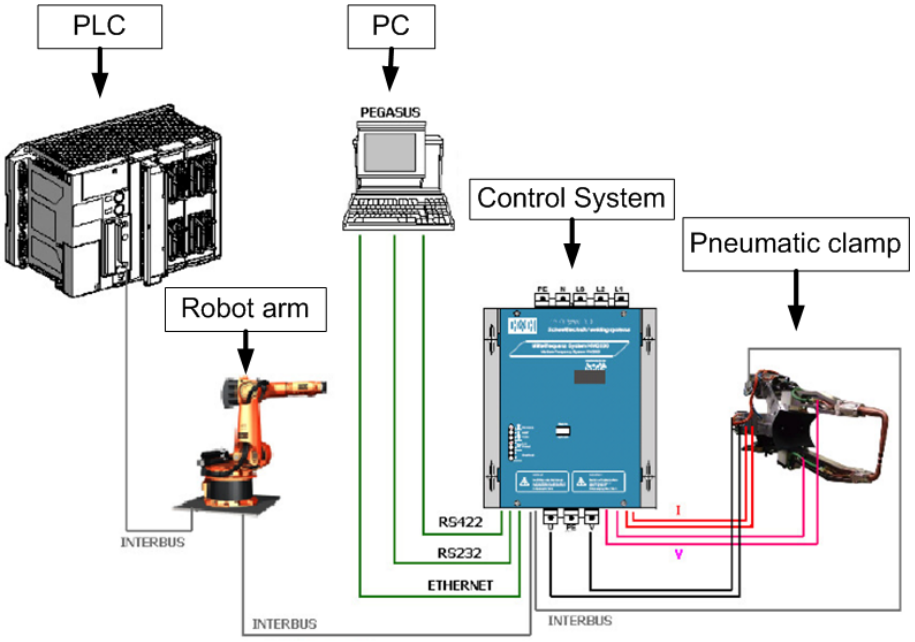


Figura 6: *Configuración Experimental de la estación de trabajo. [5].*

$$\mu = \frac{\mu}{\mu_{opt}}$$
$$\sigma = \sqrt{\frac{\sigma^2}{\mu_{opt}}}$$

Donde μ_{opt} es el total del movimiento del robot.

2.6. Patologías aplicables a las estaciones de soldadura

Existen ciertas patologías que afectan a las estaciones de soldadura que, aunque no suponen la rotura de ningún componente, en este caso los mini-términos, pueden suponer una merma en el funcionamiento de la estación que afectará en mayor o menor medida al resto de la línea. Estas patologías no son detectables por los operarios de mantenimiento ni por las alarmas de la fábrica ya que no llegan al umbral de alerta establecido. Según la experiencia de los operarios, se definieron estas 5 patologías:

Patología 1. Fallo de la válvula proporcional:

Este elemento es el responsable de proporcionar presión al cilindro, un fallo en esta pieza supondrá que la presión aplicada a la pieza no sea la correcta y la soldadura no se efectúe de forma adecuada.

Patología 2. Agarrotamiento del cilindro neumático:

El cilindro neumático es un elemento clave para la acción de soldadura, ya que es el que mantiene las piezas unidas y con la presión adecuada durante el proceso de soldadura y en el posterior proceso de enfriamiento paulatino de la pieza. Un fallo en esta pieza provocará soldaduras deficientes o inexistentes.

Patología 3. Pérdida de aislamiento en el transformador de soldadura:

Para la acción de soldadura se requiere que una corriente eléctrica circule a través de los elementos a soldar. Esto se consigue mediante un elemento llamado transformador que reduce la tensión eléctrica mientras aumenta proporcionalmente la corriente eléctrica. El fallo en este componente se produce cuando falla el aislamiento de los cables, lo que provoca una modificación en el valor de la resistencia que se traduce en una reducción de la corriente eléctrica. Este fallo puede provocar una parada en toda la línea por derivación eléctrica.

Patología 4. Pérdida de presión neumática:

La pérdida de presión en el circuito neumático provoca un mal funcionamiento del sistema. Puede estar producida por algún poro en los conductos neumáticos.

Patología 5. Pérdida de velocidad del robot:

Esta patología se produce por el desgaste en los movimientos siempre sobre los mismos ejes debido a las altas velocidades que alcanzan y a los procesos repetitivos que realizan, además de las vibraciones al transportar las piezas.

2.7. Mediciones según patologías

En este apartado se muestran las mediciones realizadas sobre la línea de soldadura sustituyendo sus piezas con elementos defectuosos que sufren las 5 patologías descritas en el apartado anterior. Estas están descritas en las Figuras 7 y 8:

	Mini-términos		
	Brazo robot (μ, σ)	Pinza de soldadura (μ, σ)	
	Posicionamiento	Movimiento	Soldadura
Patología 1	(35.55, 1.05)	(2.58, 1.24)	(24.31, 29.69)
Patología 2	(35.55, 0.80)	(8.45, 8.39)	(6.83, 14.67)
Patología 3	(35.54, 1.13)	(2.78, 1.32)	(10.57, 2.24)
Patología 4	(35.54, 0.94)	(9.35, 9.15)	(7.76, 12.46)
Patología 5	(46.33, 0.98)	(2.51, 1.12)	(8.69, 2.07)
Sin patología	(35.55, 0.67)	(2.49, 1.15)	(8.62,2.05)

Figura 7: *Medidas experimentales para cada mini-término y cada patología [5].*

	Mini-términos		
	Brazo robot (μ, σ)	Pinza de soldadura (μ, σ)	
	Posicionamiento	Movimiento	Soldadura
Patología 1	(1, 0.18)	(0.43, 0.51)	(4.05, 10.9)
Patología 2	(1, 0.13)	(1.41, 3.42)	(1.14, 5.99)
Patología 3	(1, 0.19)	(0.46, 0.54)	(1.76, 0.91)
Patología 4	(1, 0.16)	(1.56, 3.74)	(1.29, 5.08)
Patología 5	(1.30, 0.16)	(0.41, 0.46)	(1.45, 0.85)
Sin patología	(1, 0.11)	(0.42, 0.47)	(1.44, 0.84)

Figura 8: *Medidas experimentales de ciclo independiente para cada mini-término y cada patología [5].*

3. Análisis de métodos de punto de cambio

Se van a estudiar en profundidad todos los paquetes o repositorios de las librerias de R que contengan métodos de punto de cambio. Adjuntando todos los resultados de cada uno de estos algorítmos, con el fin de encontrar los más optimos para el interés de este proyecto.

3.1. Repositorio Trend [6]

Pettitt's Test

El enfoque de Pettitt (1979) se aplica para detectar un único punto de cambio en series hidrológicas o series climáticas con datos continuos, es decir, realiza una prueba no paramétrica para encontrar un cambio en la tendencia central de una serie temporal. La hipótesis H_0, sin cambio, se prueba contra la hipótesis H_1 . La estadística no paramétrica se define como:

$$K_T = max|U_{t,T}| \tag{1}$$

donde

$$U_{t,T} = \sum_{i=1}^{t} \sum_{j=t+1}^{T} sgn(X_i - X_j) \tag{2}$$

El punto de cambio de la serie se ubica en K_T, siempre que la estadística sea significativa. La probabilidad de significación de K_T se aproxima a $p \leq 0,05$ con

$$p \simeq 2\ exp\ \left(\frac{-6K_T^2}{T^3 + T^2}\right) \tag{3}$$

La prueba o test de Pettitt se lleva a cabo de la siguiente manera:

```
> require(trend)
> data(PagesData)
> pettitt.test(PagesData)

        Pettitt's test for a single change-point detection

data: PagesData
U* = 232, p-value = 0.01456
alternative hypotesis: two.sided
sample estimates:
probable change point at time K
                              17
```

Como se indica en el caso de prueba de Pettitt, con los datos de ejemplo de PagesData, el punto de cambio se encuentra en t=17 con $K_T = 232$ y p = 0.014.

A continuación se van a utilizar las mediciones realizadas por Ford. Estas mediciones son series de tiempos en segundos, en el que una máquina realiza una concreta operación. Estas series vienen en una hoja de cálculo de Excel, por lo que se deberán exportar estos datos para la posterior ejecución de la prueba de Pettitt.

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]


> pettitt <- pettitt.test(datos)
> print(pettitt)
> plot(pettitt)

        Pettitt's test for single change-point detection

data:  datos
U* = 22500, p-value < 2.2e-16
alternative hypothesis: two.sided
sample estimates:
probable change point at time K
                            150
```

Para esta muestra hemos cogido 300 datos donde en punto de cambio se situa justo en medio de la muestra, por lo que el punto de cambio debería ubicarse en la posición 150. En este caso el test de Pettitt nos ha dado un resultado de k=150 con $K_T = 22500$ y un p-value $< 2{,}2e^{-16}$.

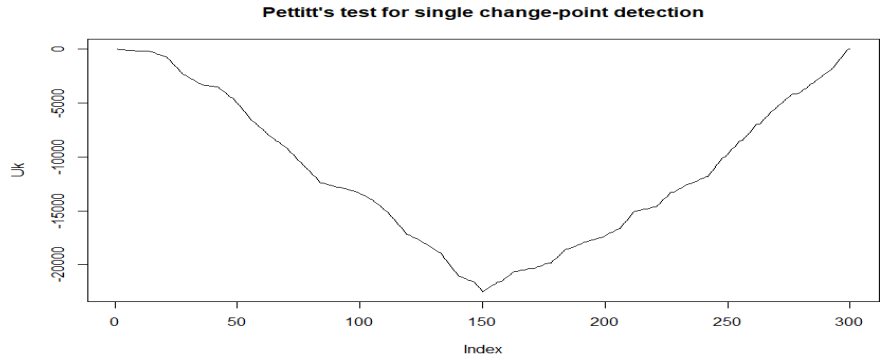


Figura 9: *Plot del resultado del test de Pettitt.*

Buishand Range Test (1982)

La X indica una variable aleatoria normal, en el siguiente modelo con un solo cambio se puede proponer:

$$X_i = \begin{cases} \mu + \epsilon_i & i = 1, ..., m \\ \mu + \Delta + \epsilon_i & i = m+1, ..., n \end{cases} \tag{4}$$

$\epsilon \approx N(0, \sigma)$. La hipótesis nula $\Delta = 0$ se compara con la hipótesis alternativa $\Delta \neq 0$.

En el test o prueba de rango de Buishand, las sumas parciales reescaladas y ajustadas se calculan como

$$S_k = \sum_{i=1}^{k} (x_i - \hat{x}) \tag{5}$$

$$(1 \leq i \leq n) \tag{6}$$

La prueba estadística se calcula como:

$$Rb = \frac{maxS_k - minS_k}{\sigma} \tag{7}$$

el p.valor se estima con una simulación de Monte Carlo utilizando m réplicas.

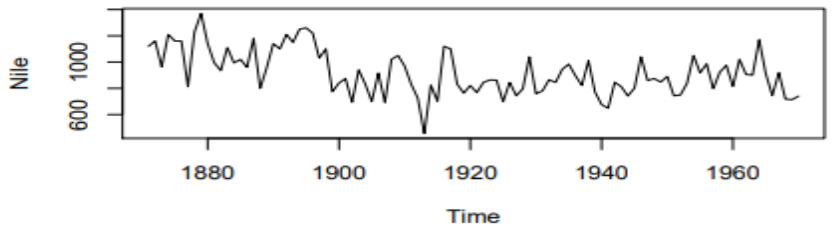
La prueba de rango de Buishand se lleva a cabo de la siguiente manera:

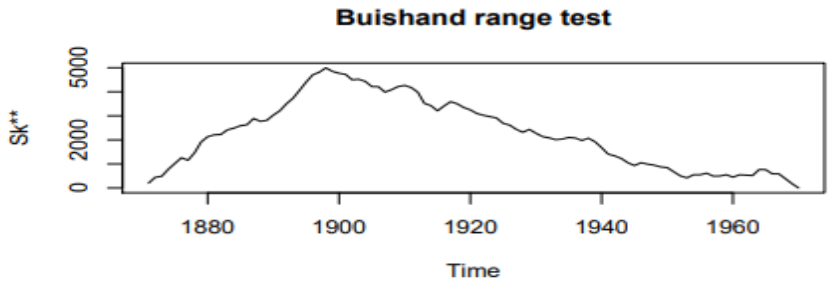
```
> require(trend)
> (res <- br.test(Nile))

                Buishand range test

data: Nile
R / sqrt(n) = 2.9518, n = 100, p-value < 2.2e-16
alternative hypothesis: true delta is not equal to 0
sample estimates:
probable change point at time K
                                28

> plot(Nile); plot(res)
```





Para la muestra de 300 datos de Ford:

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]

>buishand <- br.test(datos)
>print(buishand)
>plot(buishand)

        Buishand range test

data:  datos
R / sqrt(n) = 8.4412, n = 300, p-value < 2.2e-16
alternative hypothesis: true delta is not equal to 0
sample estimates:
probable change point at time K
                            150
```

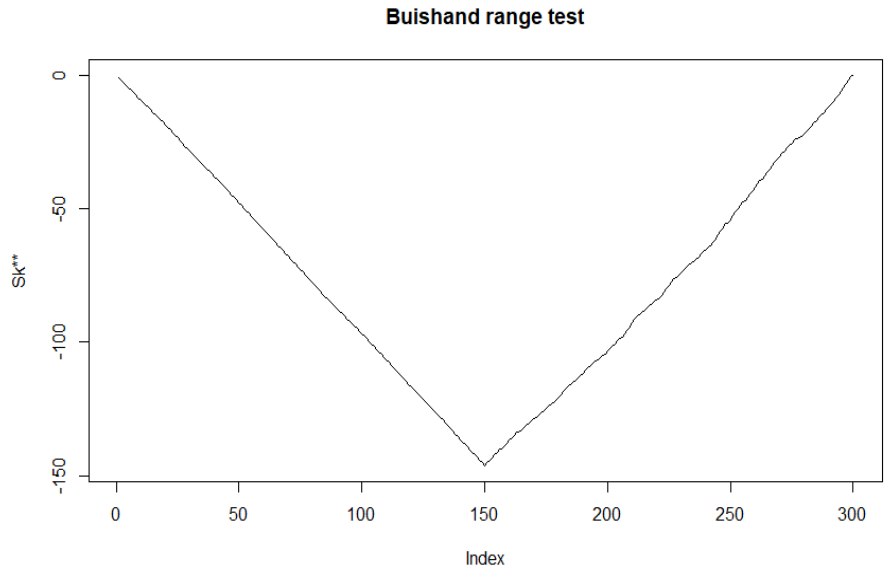


Figura 10: *Plot del resultado del test de Buishand.*

Buishand U Test (1984)

En la prueba U de Buishand, la hipótesis nula es la misma que en la prueba de rango de Buishand (ver la Ecuación nº4). La estadística es

$$U = [n(n+1)]^{-1} \sum_{k=1}^{n-1} (S_k/D_x)^2 \tag{8}$$

donde

$$D_x = \sqrt{n^{-1} \sum_{i=1}^{n} (x_i - \bar{x})} \tag{9}$$

y S_k como se indica en la Ecuación nº5. El p.valor se estima con una simulación de Monte Carlo utilizando m réplicas.

La prueba U de Buishand se lleva a cabo de la siguiente manera:

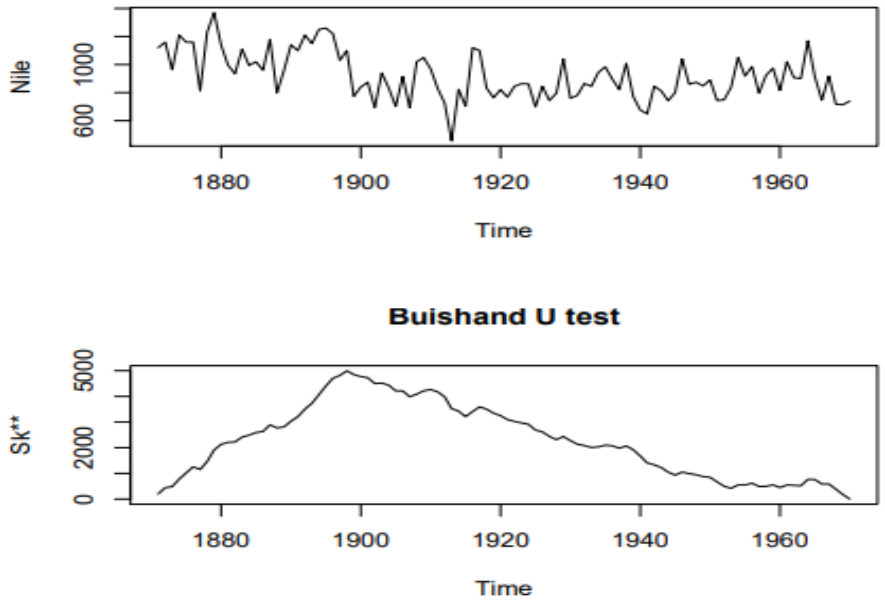
```
> require(trend)
> (res <- bu.test(Nile))

        Buishand U test

data: Nile
U = 2.4764, n = 100, p-value < 2.2e-16
alternative hypothesis: true delta is not equal to 0
```

```
sample estimates:
probable change point at time K
                                28

> par(mfrow=c(2,1))
> plot(Nile); plot(res)
```



Para la muestra de 300 datos de Ford:

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]

>buishandU <- bu.test(datos)
>print(buishandU)
>plot(buishandU)

        Buishand U test

data:  datos
U = 24.444, n = 300, p-value < 2.2e-16
alternative hypothesis: true delta is not equal to 0
```

```
sample estimates:
probable change point at time K
                              150
```

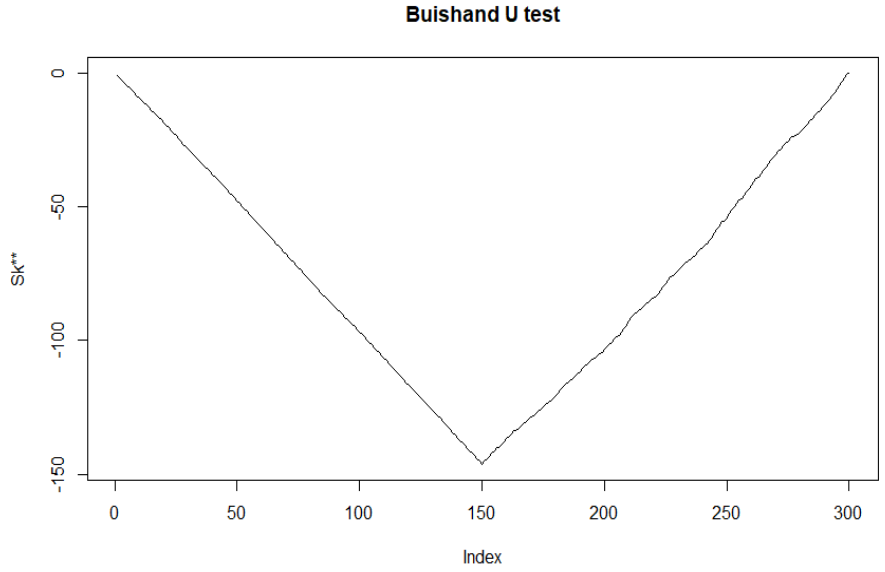


Figura 11: *Plot del resultado del test U de Buishand.*

Standard Normal Homogeneity Test

En la "Standard Normal Homogeneity Test.º prueba estándar de homogeneidad normal, la hipétesis nula es la misma que en la prueba de rango de Buishand (ver la Ecuación nº4). La prueba estadística es:

$$T_k = kz_1^2 + (n-k)z_2^2 \tag{10}$$

$$(1 \leq k < n) \tag{11}$$

donde

$$z_1 = \frac{1}{k}\sum_{i=1}^{k}\frac{x_i - \bar{x}}{\sigma} \tag{12}$$

$$z_2 = \frac{1}{n-k}\sum_{i=k+1}^{n}\frac{x_i - \bar{x}}{\sigma} \tag{13}$$

El valor crítico es:

$$T = maxT_k \tag{14}$$

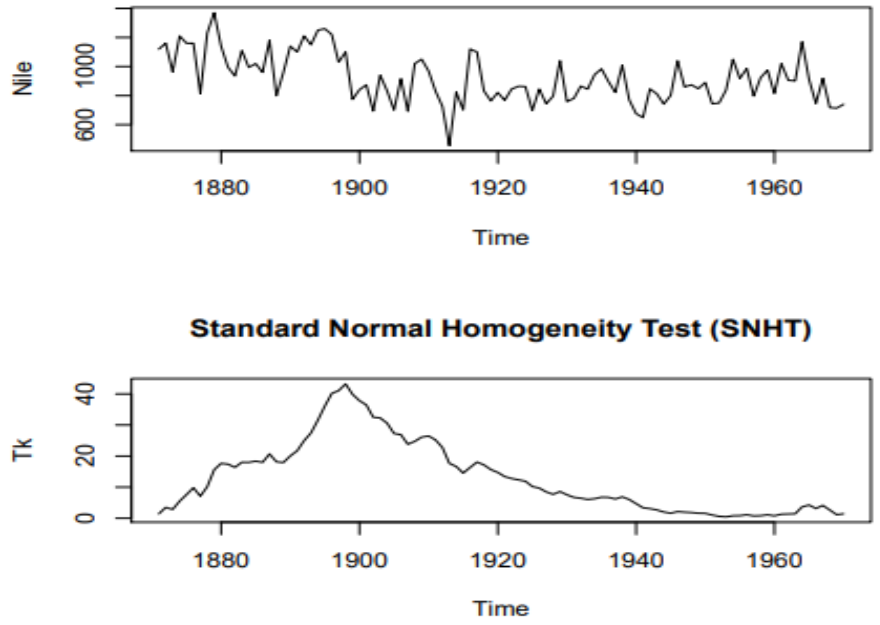
La prueba estándar de homogeneidad normal se lleva a cabo de la siguiente manera:

```
> require(trend)
> (res <- snh.test(Nile))

                Standard Normal Homogeneity Test (SNHT)

data: Nile
T = 43.219, n = 100, p-value < 2.2e-16
alternative hypothesis: true delta is not equal to 0
sample estimates:
probable change point at time K
                                28

> par(mfrow=c(2,1))
> plot(Nile); plot(res)
```



Para la muestra de 300 datos de Ford:

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
```

```
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]

>snht <- snh.test(datos)
>print(snht)
>plot(snht)

        Standard Normal Homogeneity Test (SNHT)

data:  datos
T = 285.01, n = 300, p-value < 2.2e-16
alternative hypothesis: true delta is not equal to 0
sample estimates:
probable change point at time K
                            150
```

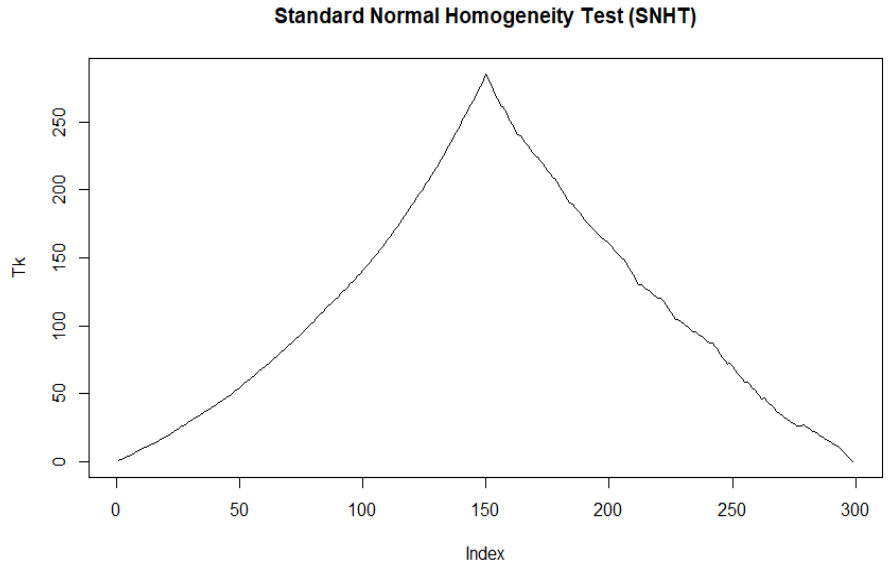


Figura 12: *Plot del resultado de la prueba estándar de homogeneidad normal.*

3.2. Repositorio Strucchange [7]

F tests

Un enfoque diferente para investigar si la hipótesis nula "sin cambio estructural"se sostiene, es utilizar las estadísticas de la prueba F o F tests. Una diferencia importante es que se especifica la alternativa: mientras que las pruebas de fluctuación generalizadas son adecuadas para varios patrones de cambios estructurales, las pruebas F estan diseñadas para probar contra una alternativa de turno único. De este modo, la alternativa puede formularse sobre la base del modelo

$$\beta_i = \begin{cases} \beta_A & (1 \leq i \leq i_0) \\ \beta_B & (i_0 < i \leq n) \end{cases} \tag{15}$$

donde i_0 es algún punto de cambio en el intervalo $(k, n-k)$. Chow (1960) fue el primero en sugerir una prueba de este tipo sobre el cambio estructural en el caso de que se conozca el punto de cambio i_0. Propuso ajustar dos regresiones separadas para las dos submuestras definidas por i_0 y rechazar cada vez que

$$F_{i_0} = \frac{\hat{u}^\top \hat{u} - \hat{e}^\top \hat{e}}{\hat{e}^\top \hat{e}/(n-2k)} \tag{16}$$

donde $\hat{e} = (\hat{u}_A, \hat{u}_B)^\top$ son los residuales del modelo completo, donde los coeficientes en las submuestras se estiman por separado, y $\hat{u}$ son los residuales del modelo restringido, donde los parámetros se ajustan una vez para todas las observaciones. La prueba estadística F_{i_o} tiene una distribución X^2 asintótica con k grados de libertad y Fi_0/k tiene una distribución F exacta con k y $n-2k$ grados de libertad. El principal incoveniente de esta prueba de Chow es que el punto de cambio debe conocerse de antemano, pero hay pruebas basadas en las estadísticas de F, que no requieren una especificaión de un punto de cambio en particular, como la función Fstats o F statistics que definiremos a continuación.

F statistics: función Fstats

Una idea natural para extender las ideas de la prueba de Chow es calcular las estadísticas de F para todos los puntos de cambio potenciales o para todos los puntos de cambio potenciales en un intervalo $[\underline{i}, \bar{i}]$ y rechazar si alguna de esas estadísticas es demasiado grande. Por lo tanto, el primer paso es calcular las estadísticas F de F_i para $k < \underline{i} \leq i \leq \bar{i} < n-k$ lo cual se puede hacer fácilmente usando la función Fstats. Una vez más, el modelo a probar se especifiva mediante una interfaz de fórmula y los parámetros $\underline{i}$ y $\bar{i}$ están represantados por desde y hasta, respectivamente.
Esta función devuelve un objeto de clase Fstats que contiene principalmente una serie temporal de F estadísticas. De manera que estos objetos se pueden imprimir, trazar, probar, etc. A continuación unas líneas de código de como funciona este test.

```
## Nile data with one breakpoint: the annual flows drop in 1898
## because the first Ashwan dam was built
data("Nile")
plot(Nile)

## test the null hypothesis that the annual flow remains constant
## over the years
fs.nile <- Fstats(Nile ~ 1)
plot(fs.nile)
sctest(fs.nile)
## visualize the breakpoint implied by the argmax of the F statistics
plot(Nile)
lines(breakpoints(fs.nile))
```

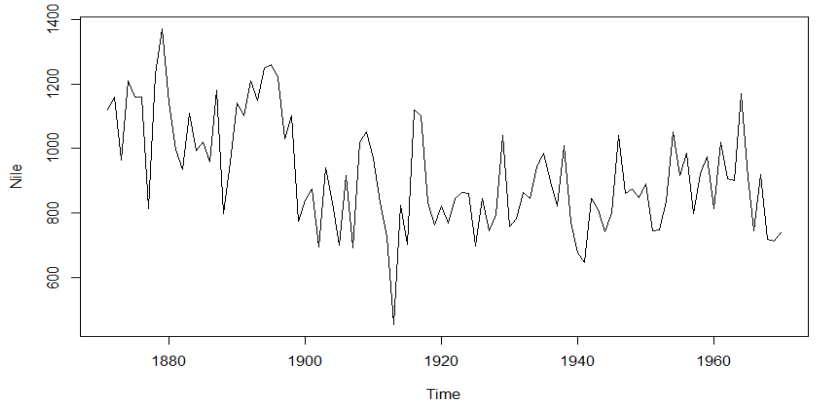


Figura 13: *Plot de los datos de ejemplo Nile.*

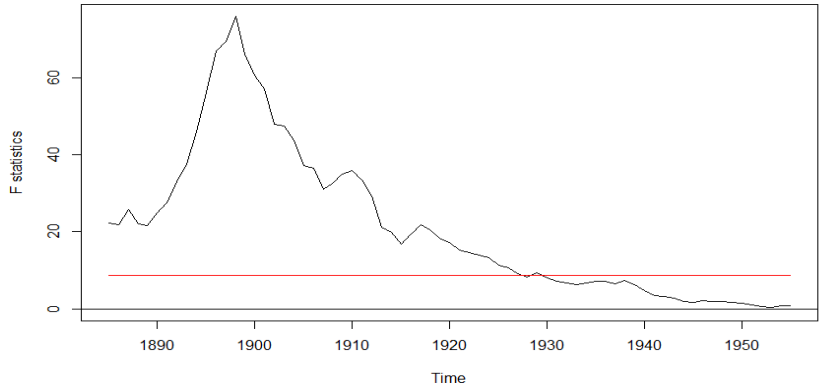


Figura 14: *Plot de Fstats de Nile.*

Sctest es una función genérica para realizar o extraer pruebas de cambio estructural basadas en varios tipos de objetos. Podemos ver el ejemplo con los datos Nile en la Figura 15

```
> sctest(fs.nile)

        supF test

data:  fs.nile
sup.F = 75.93, p-value = 2.22e-16
```

Figura 15: *Sctest de Nile*

La función breakpoints calcula los puntos de ruptura en las relaciones de regresión. Dada una serie de interrupciones, la función calcula los puntos de interrupción óptimos. Como se ve en la Figura 16 detecta como punto de cambio la observación número 28.

```
> breakpoints(fs.nile)

        Optimal 2-segment partition:

Call:
breakpoints.Fstats(obj = fs.nile)

Breakpoints at observation number:
28

Corresponding to breakdates:
1898
```

Figura 16: *Breakpoints de Nile*

A continuación se muestra el trozo de código con los 300 datos extraidos de una estación de soldadura de Ford, donde el punto de cambio se produce en la observación 150.

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]
##cambio de vector numerico a serie temporal
>datos_ts <- ts(datos)

> fs.datos <- Fstats(datos_ts ~ 1)
> plot(fs.datos)
> sctest(fs.datos)

        supF test

data:  fs.datos
sup.F = 6072.9, p-value < 2.2e-16

> breakpoints(fs.datos)

         Optimal 2-segment partition:

Call:
breakpoints.Fstats(obj = fs.datos)

Breakpoints at observation number:
150

Corresponding to breakdates:
150
```

La observación del punto de cambio es en el número 150 como se preveía anteriormente, con un p-valor$< 2{,}2e-16$. También se puede ver el resultado en la Figura 17.

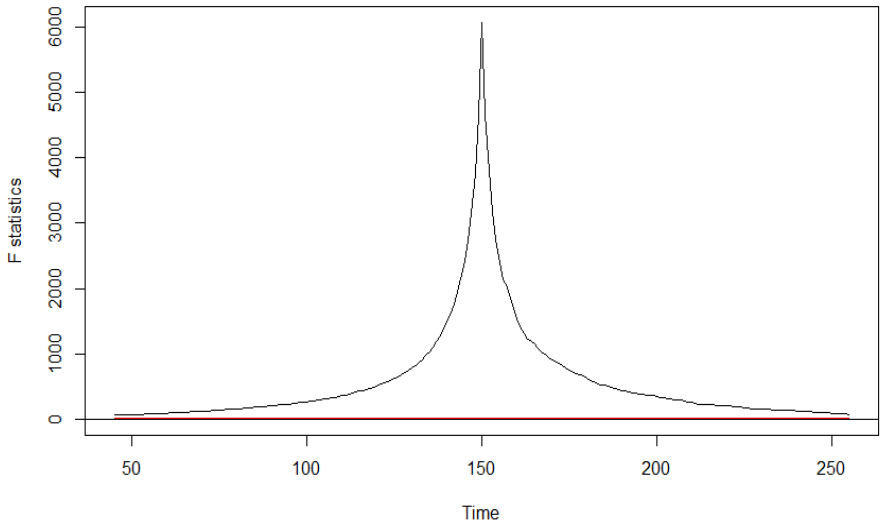


Figura 17: *Fstats de los datos de Ford*

Empirical fluctuation precesses: función efp

Dada una fórmula que describe un modelo de regresión lineal que se va a probar, la función efp crea un objeto de la clase "efp" que contiene un proceso de fluctuación empírica ajustado de un tipo específico.

Procesos CUSUM: El primer tipo de procesos que se pueden calcular son procesos CUSUM, que contienen sumas acumuladas de residuos estandarizados. Existen dos tipos de procesos CUSUM el "$Rec-CUSUM$" y el "$OLS-CUSUM$", estos están disponibles en la función efp especificando el tipo de argumento "type". La estadística se puede observar en el paper (strucchange: An R Package for Testing for Structural Change in Linear Regression Models, p.5 [7]).
De este modo, vamos a ver el código y resultado de los dos procesos de CUSUM con los datos de Ford. Primero vamos con el proceso OLS-CUSUM:

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]
##cambio de vector numerico a serie temporal
>datos_ts <- ts(datos)

> efp <- efp(datos_ts ~ 1, type = "OLS-CUSUM")
> plot(efp)
> sctest(efp)

        OLS-based CUSUM test

data:  efp
S0 = 8.4412, p-value < 2.2e-16
```

En el gráfico de la Figura 18 se puede observar como el punto de cambio se encuentra en la posición 150 con un p-valor$< 2{,}2e-16$.

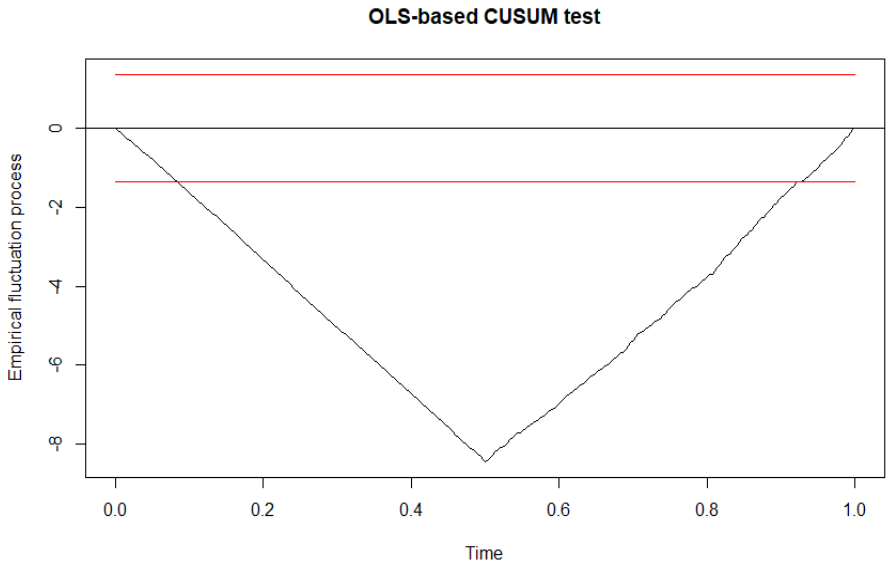


Figura 18: *Proceso OLS-CUSUM con los datos de Ford*

Se va a seguir con la prueba ahora con el segundo tipo de proceso CUSUM, el Rec-CUSUM. Para esto sólo hay que cambiar el type="$Rec - CUSUM$" en el argumento de la función.

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]
##cambio de vector numerico a serie temporal
>datos_ts <- ts(datos)

> efp1 <- efp(datos_ts ~ 1, type = "Rec-CUSUM")
> plot(efp1)
> sctest(efp1)

        Recursive CUSUM test

data:  efp1
S = 5.2984, p-value < 2.2e-16
```

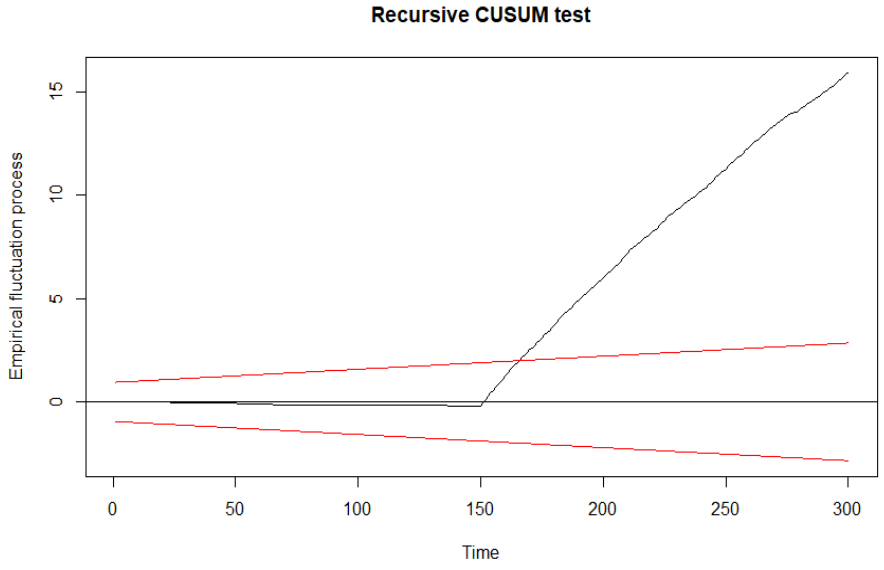


Figura 19: *Proceso Rec-CUSUM con los datos de Ford*

Procesos MOSUM: Otra posibilidad para detectar un cambio estructural es analizar sumas móviles de residuos (en lugar de usar sumas acumuladas de los mismos residuos). En MOSUM también existen dos tipos cómo en CUSUM, esta el Rec-MOSUM y el OLS-MOSUM. La estadística se puede observar en el paper

(strucchange: An R Package for Testing for Structural Change in Linear Regression Models, p.5 [7]).

A continuación, se muestra el código y resultado de los procesos OLS-MOSUM y Rec-MOSUM respectivamente:

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]
##cambio de vector numerico a serie temporal
>datos_ts <- ts(datos)

> efp2 <- efp(datos_ts ~ 1, type = "OLS-MOSUM")
> plot(efp2)
> sctest(efp2)


        OLS-based MOSUM test

data:  efp2
M0 = 2.8345, p-value = 0.01
```

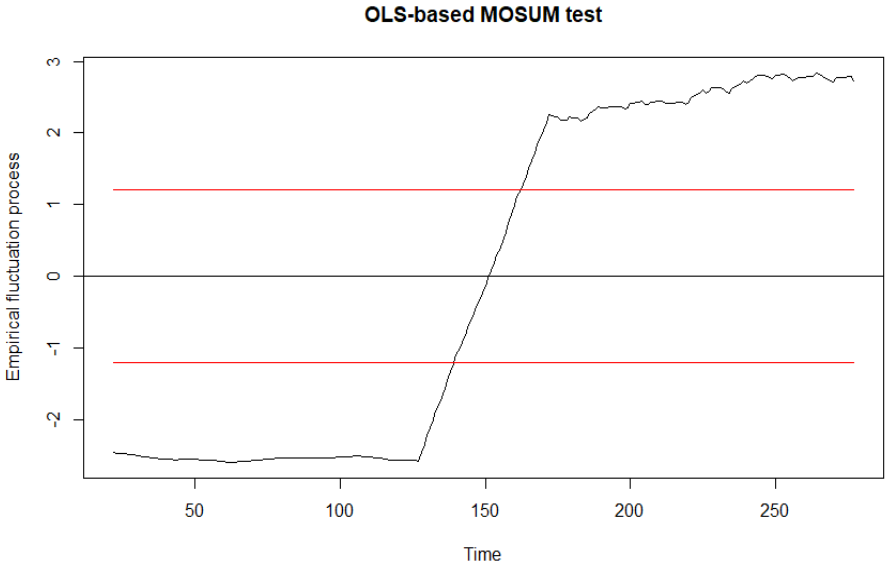


Figura 20: *Proceso OLS-MOSUM con los datos de Ford*

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]
##cambio de vector numerico a serie temporal
>datos_ts <- ts(datos)

> efp3 <- efp(datos_ts ~ 1, type = "Rec-MOSUM")
> plot(efp3)
> sctest(efp3)


        Recursive MOSUM test

data:  efp3
M = 5.5759, p-value = 0.01
```

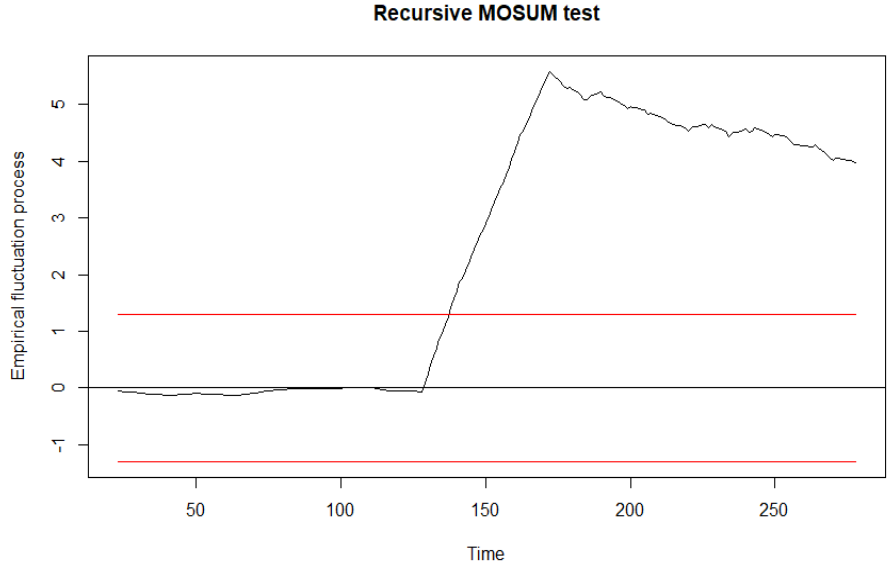


Figura 21: *Proceso Rec-MOSUM con los datos de Ford*

3.3. Repositorio wbsts [8]

The Wild Binary Segmentation Algorithm

The Wild Binary Segmentation Algorithm o Algoritmo de Segmentación Binaria Salvaje es una técnica para la estimación del número y ubicaciones de los puntos de cambio en la estructura de segundo orden de una serie temporal. El núcleo del procedimiento de segmentación es el método de Segmentación Binaria Salvaje (WBS), una técnica con un cierto mecanismo aleatorio. La ventaja de WBS sobre la segmentación binaria estándar radica en su función de localización, a la cual funciona en los casos en que las distancias entre los puntos de cambio son cortas. Además, no restringe el número total de puntos de cambio que puede tener una serie temporal. El modelo estadístico se puede ver en (Multiple Change-point Detection for non-stationary Time Series Using Wild Binary Segmentation, p.6 [8]).
Se puede estimar el punto de cambio via la segmentación binaria normal o via la segmentación binaria salvaje cambiando el parámetro M=1/0. M=1 para BS y M=0 para WBS. A continuación se muestra el funcionamiento del algoritmo para la muestra de los datos de Ford.

Binary Segmentation

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]
##cambio de vector numerico a serie temporal
>datos_ts <- ts(datos)

##Estimate the change points via Binary Segmentation

> bs <- wbs.lsw(datos_ts,M=1)
> print(bs)

$`cp.bef`
[1] 147

$cp.aft

147
```

cp.bef devuelve el punto de cambio estimado antes des post-procesado y el cp.aft devuelve el punto de cambio estimado después del post-procesado. Añadiendo cp.aft al final de la línea de código del algoritmos podemos extraer únicamente el punto de cambio estimado después del post-procesado. Así, como podemos observar a continuación, el algorítmo devuelve 147 como estimación del punto de cambio.

```
> wbs.lsw(datos_ts,M=1)$cp.aft

147
```

Wild Binary Segmentation

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]
##cambio de vector numerico a serie temporal
>datos_ts <- ts(datos)

##Estimate the change points via Wild Binary Segmentation

> wbs <- wbs.lsw(datos_ts,M=0)
> print(wbs)

$`cp.bef`
[1] 147  90 186  49 120 265  71 106 280

$cp.aft
[1] 147

> wbs.lsw(datos_ts,M=0)$cp.aft

147
```

El WBS al tener un cierto mecanismo aleatorio, en cada ejecución devuelve una serie de puntos de cambio antes del post-procesado de forma aleatoria. Dando así, un resultado diferente para cada ejecución.

3.4. Repositorio wbs

El siguiente repositorio ofrece las técnicas de segmentación binaria estandar y salvaje. En este caso, son una técnica para la estimación del número y ubicación de múltiples puntos de cambio en datos númericos y no en series temporales como hemos descrito en el punto anterior. Con la muestra de los datos de Ford vamos a ver a continuación el funcionamiento de estos dos algoritmos.

Binary Segmentation para identificar ubicaciones potenciales de los puntos de cambio en la media del vector de entrada. El objeto devuelto se puede pasar a la función changepoints, que encuentra la estimación final de los puntos de cambio en función del umbral.

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]

##Estimate the change points via Binary Segmentation

> s <- sbs(datos)
> plot(s)
> changepoints(s)
```

```
> s <- sbs(datos)
> plot(s)
> changepoints(s)
$`sigma`
[1] 0

$th
[1] 0

$no.cpt.th
[1] 279

$cpt.th
$cpt.th[[1]]
  [1] 150 242 212 276 263 256 227 269 249 248 293 299 156 164 221 235 185 241 184 206 228 205 198 177 220 170 279 213 178
 [30] 163 255 262  21 199 234 171 157 105 191  28 133  42 277 140 119  84 147  35  14  98 292   7  70  63 112 126  49  56
 [59] 278 290 286 291 289 155  97 288 254 226 275 211  96 162 261 284 174 183 204 233 176 247 268 297 190 202 224 231 246
 [88] 287  77 181 252 272  80 131 132  92 209 273  13  79 103 124 153 195 210 217 259 260 267 285 298  12  78 168 175 182
[117] 194 218 239 245 274 296  82  91 111  26  67  75  90 104 125 129 138 154 188 189 203 216 225 232 253 266  10  17  18
[146]  53  59  81 109 110 116   3   4   5  25  34  40  47  52  62  68  74  87 115 130 139 159 161 166 193 196 201 215 230
[175] 238 244 251 258 281 283 295  11  19  24  27  38  39  45  60  66  73  76 101 108 122  30  32  33  37  46  58  86  89
[204]  93  95 102 107 123 149 151 152 158 160 172 173 179 180 186 187 192 197 200 207 208 214 219 222 223 229 236 237 240
[233] 243 250 257 264 265 270 271 280 282 294   2   8   9  29  31  43  44  48  50  51  54  55  57  61  64  65  69  83  85
[262]  88  94  99 100 106 113 114 117 118 120 121 134 135 136 137 142 143 145


$Kmax
[1] 279
```

- **no.cpt.th** devuelve el número de puntos de cambio detectados.
- **cpt.th** devuelve una lista de las ubicaciones de los puntos de cambio detectados.

- **Kmax** devuelve el número máximo de puntos de cambio detectados.

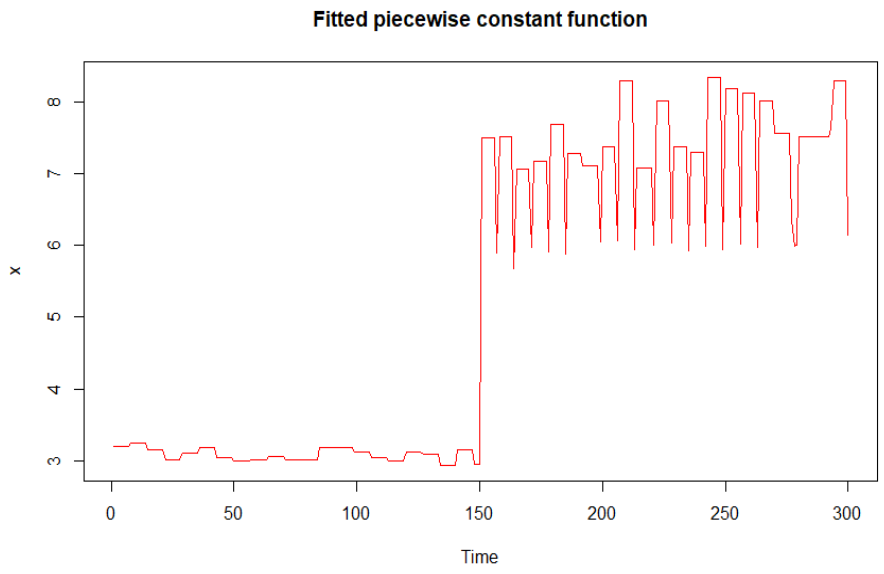


Figura 22: *Plot Binary Segmentation*

Se puede observar tanto en la gráfica 22 como en la lista devuelta por el algoritmo que el cambio de punto mas relevante se encuentra en el 150.

Wild Binary Segmentation para identificar ubicaciones potenciales de los puntos de cambio en la media del vector de entrada. El objeto devuelto se puede pasar a la función changepoints, que encuentra la estimación final de los puntos de cambio en función de los criterios de parada elegidos.

```
##lectura del excel
> muestras <- read_excel("muestras.xlsx", sheet = "Hoja1")
##seleccion de la columna 3 'mini-term'
> filtro <- muestras[,3]
##cambio de lista a vector numerico
> datos <- as.numeric(unlist(filtro))
##seleccion de los 300 datos con el change point en el 150
> datos <- datos[2888:3187]

##Estimate the change points via Wild Binary Segmentation

> w <- wbs(datos)
> plot(w)
> changepoints(w)
```

Se va a desglosar el resultado de changepoints(w) para su perfecta visualización.

- **no.cpt.th** devuelve el número de puntos de cambio detectados.

```
$no.cpt.th
[1] 275
```

- **cpt.th** devuelve una lista de las ubicaciones de los puntos de cambio detectados.

```
$cpt.th
$cpt.th[[1]]
  [1] 150 242 276 206 279 212 248 249 299 262 255 227 256 221 263 178 293 156 163 184 164 185 228 235 199 241 205 234 171
 [30] 198 177 213 157 220 170 269  21  84 105  42 140 191  28 133 277 147 119  35  14  63  98 292  70   7  49 112 126  56
 [59] 278 290 286 287 155  97 254 226 275 211  96 162 261 284 285 174 175 183 204 233 247 268 297 298 190 202 224 231 246
 [88]  77  78  95 181 252 253 272 131 132  89  94 209 273  13 103 124 153 195 210 217 259 260 267  12 168 182 194 218 239
[117] 245 274 296  88  93 111  67  83  87  92 104 125 129 130 138 143 144 154 188 189 203 216 225 232 266  10  17  20  53
[146]  54  59 109 110 116   3   4   5  25  26  34  40  47  52  62  68  74  82 115 139 159 161 166 193 196 201 215 230 238
[175] 244 251 258 281 283 295  11  24  38  39  45  60  66  73  81 101 108 122  30  32  33  37  46  58  86  91 102 107 123
[204] 149 151 152 158 160 172 173 176 179 180 186 187 192 197 200 207 208 214 219 222 223 229 236 237 240 243 250 257 264
[233] 265 270 271 280 282 288 289 291 294   2   8   9  27  29  31  43  44  48  50  51  55  57  61  64  65  69  85  90  99
[262] 100 106 113 114 117 118 120 121 134 135 136 137 142 146
```

- **Kmax** devuelve el número máximo de puntos de cambio detectados.

```
$Kmax
[1] 50
```

- **ic.curve** devuelve una lista con los valores de los criterios de información elegidos.

```
$ic.curve
$ic.curve$`ssic.penalty`
 [1]   239.4403  -214.6171  -226.1170  -224.9694  -221.6827  -233.3991  -246.2194  -246.1794  -252.1588  -255.7875
[11]  -253.0789  -249.0451  -244.9228  -255.3567  -274.0642  -285.2147  -280.2505  -290.3198  -292.5959  -292.8679
[21]  -300.1461  -307.1780  -315.1326  -320.5950  -314.9150  -309.4656  -319.7202  -331.9023  -351.9367  -346.5992
[31]  -360.6898  -381.5608  -396.0254  -458.8114  -498.1361  -555.5378  -612.7280  -651.7890  -652.7682  -680.9944
[41]  -694.5912  -693.5500  -712.9933  -736.8792  -766.3174  -804.5280  -861.2678  -919.8978  -949.8290  -984.5570
[51] -1009.1762

$ic.curve$bic.penalty
 [1]   239.4403  -214.7173  -226.3174  -225.2700  -222.0835  -233.9000  -246.8205  -246.8807  -252.9602  -256.6891
[11]  -254.0807  -250.1471  -246.1249  -256.6591  -275.4667  -286.7174  -281.8534  -292.0229  -294.3992  -294.7713
[21]  -302.1497  -309.2818  -317.3366  -322.8992  -317.3194  -311.9701  -322.3248  -334.6071  -354.7418  -349.5044
[31]  -363.6952  -384.6664  -399.2311  -462.1173  -501.5422  -559.0441  -616.3345  -655.4956  -656.5750  -684.9015
[41]  -698.5984  -697.6573  -717.2009  -741.1869  -770.7253  -809.0361  -865.8761  -924.6062  -954.6377  -989.4659
[51] -1014.1852

$ic.curve$mbic.penalty
 [1]  239.4403 -212.5585 -222.3728 -220.0034 -215.2734 -226.6074 -238.7232 -237.9845 -244.0823 -247.8355 -244.2730
[12] -239.7531 -234.7235 -245.3347 -263.5019 -274.7896 -268.6061 -278.0580 -279.6590 -279.2497 -285.8528 -293.0194
[23] -301.0974 -306.6945 -300.4883 -294.3689 -304.8006 -317.1600 -337.3717 -331.5080 -345.7358 -366.7841 -381.4077
[34] -444.3710 -483.8626 -541.4416 -598.1457 -635.8600 -635.2028 -662.1985 -674.5759 -672.6092 -691.5664 -714.7822
[45] -743.4592 -781.9727 -838.4418 -896.1989 -925.6040 -959.6620 -983.6111
```

- **cpt.ic** devuelve una lista con los puntos de cambio detectados para cada criterio de información considerado.

```
$cpt.ic
$cpt.ic$`ssic.penalty`
 [1] 150 242 276 206 279 212 248 249 299 262 255 227 256 221 263 178 293 156 163 184 164 185 228 235 199 241 205 234 171
[30] 198 177 213 157 220 170 269  21  84 105  42 140 191  28 133 277 147 119  35  14  98

$cpt.ic$bic.penalty
 [1] 150 242 276 206 279 212 248 249 299 262 255 227 256 221 263 178 293 156 163 184 164 185 228 235 199 241 205 234 171
[30] 198 177 213 157 220 170 269  21  84 105  42 140 191  28 133 277 147 119  35  14  98

$cpt.ic$mbic.penalty
 [1] 150 242 276 206 279 212 248 249 299 262 255 227 256 221 263 178 293 156 163 184 164 185 228 235 199 241 205 234 171
[30] 198 177 213 157 220 170 269  21  84 105  42 140 191  28 133 277 147 119  35  14  98
```

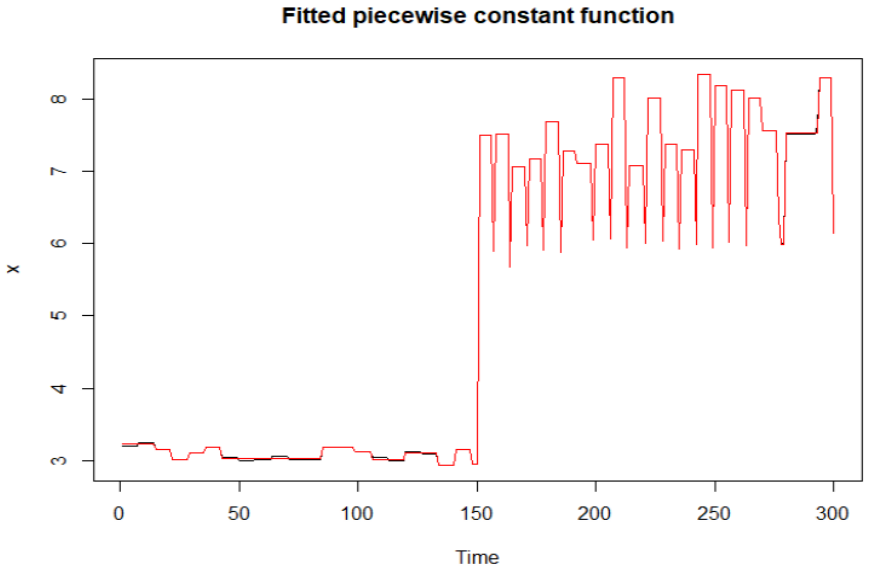


Figura 23: *Plot Binary Segmentation*

Como se puede observar en los resultados obtenidos por el algoritmo de segmentación binaria salvaje este extraen 50 puntos potenciales para cada criterio de información. Estos estan ordenador por orden de prioridad, se pueden ver en el apartado de cpt.ic.

3.5. Repositorio cpm

El package cpm o change point model es una implementación de varios modelos diferentes de puntos de cambio para realizar la detección de cambios tanto paramétricos como no paramétricos en flujos de datos univariados.
Este paquete implementa los change points model utilizando diferentes pruebas estadísticas que se pueden elegir para optimizar la detección del punto de cambio:

- *Student* Prueba estadística t de Student. Se utiliza para detectar cambios de media en una secuencia Gaussiana.
- *Bartlett* Prueba estadística de Bartlett. Se utiliza para detectar cambios de varianza en una secuencia Gaussiana.
- *GLR* Prueba estadística de la Relación de Probabilidad Generalizada . Se utiliza para detectar cambios de media y varianza en una secuencia Gaussiana.
- *Exponential* Prueba estadística de la Relación de Probabilidad Generalizada para la distribución exponencial. Se utiliza para detectar cambios en el parámetro de una secuencia distribuida exponencialmente.
- *FET* Prueba estadística exacta de los Pescadores . Se utiliza para detectar cambios de parámetros en una secuencia de Bernoulli.
- *Mann-Whitney* Prueba estadística de Mann-Whitney . Se utiliza para detectar cambios de ubicación en una secuencia con una distribución no Gaussiana.
- *Mood* Prueba estadística de Mood . Se utiliza para detectar cambios de escala en una secuencia con una distribución no Gaussiana.
- *Lepage* Prueba estadística de Lepage. Se utiliza para detectar la ubicación y los cambios en una secuencia con una distribución no Gaussiana.
- *Kolmogorov-Smirnov* Prueba estadística de Kolmogorov-Smirnov. Se utiliza para detectar cambios arbitrarios en una secuencia con una distribución no Gaussiana.
- *Cramer-von-Mises* Prueba estadística de Cramer-von-Mises. Se utiliza para detectar cambios arbitrarios en una secuencia con una distribución no Gaussiana.

Para obtener una descripción más completa de estas estadísticas consultar el manual completo del paquete [9].

CPM es un enfoque para detección de cambios secuenciales que permite que las pruebas de hipótesis estadísticas se implementen de manera secuencial. Las dos funciones principales del repositorio son detectChangePoint para detectar puntos de cambio únicos, y processStream para detectar múltiples puntos de cambio. Para la detección de puntos de cambio únicos existen dos tipos principales de problemas de detección de cambios, por lotes (batch) y secuenciales.

Detección por lotes o 'batch'

En este caso, hay una secuencia de longitud fija que consta de n observaciones de las variables aleatorias, y se quiere probar si esta secuencia contiene puntos de cambio. Este tipo de detección de cambios es retrospectivo, lo que significa que la decisión de si se ha producido un cambio en un punto en particular de la secuencia se realiza utilizando todas las observaciones disponibles, incluidas aquellas que se producen más adelante en la secuencia. Estos métodos por lotes funcionan bien cuando solo hay un pequeño número de puntos de cambio, pero pueden ser inviables computacionamente cuando hay un mayor número de puntos de cambio. En el package cpm la función por lotes se escribe de la siguiente manera: detectChangePointBatch. A continuación se verá su funcionamiento con los datos de prueba de Ford para cada tipo de estadística descrita anteriormente:

Student > detectChangePointBatch(datos,"Student",alpha = 0.05)

```
$changePoint
[1] 150

$changeDetected
[1] TRUE

$alpha
[1] 0.05

$threshold
[1] 3.176664
```

Bartlett > detectChangePointBatch(datos,"Bartlett",alpha = 0.05)

```
$changePoint
[1] 2

$changeDetected
[1] TRUE

$alpha
[1] 0.05

$threshold
[1] 9.955205
```

GLR > detectChangePointBatch(datos,"GLR",alpha = 0.05)

```
$changePoint
[1] 2

$changeDetected
[1] TRUE

$alpha
[1] 0.05

$threshold
[1] 13.44508
```

Exponential > detectChangePointBatch(datos,"Exponential",alpha = 0.05)

```
$changePoint
[1] 150

$changeDetected
[1] TRUE

$alpha
[1] 0.05

$threshold
[1] 10.35058
```

Mann-Whitney > detectChangePointBatch(datos,"Mann-Whitney",alpha = 0.05)

```
$changePoint
[1] 150

$changeDetected
[1] TRUE

$alpha
[1] 0.05

$threshold
[1] 3.086255
```

Mood > detectChangePointBatch(datos,"Mood",alpha = 0.05)

```
$changePoint
[1] 242

$changeDetected
[1] TRUE

$alpha
[1] 0.05

$threshold
[1] 3.124458
```

Lepage > detectChangePointBatch(datos,"Lepage",alpha = 0.05)

```
$changePoint
[1] 150

$changeDetected
[1] TRUE

$alpha
[1] 0.05

$threshold
[1] 13.30759
```

Kolmogorov-Smirnov > detectChangePointBatch(datos,"Kolmogorov-Smirnov",alpha = 0.05)

```
$changePoint
[1] 60

$changeDetected
[1] TRUE

$alpha
[1] 0.05

$threshold
[1] 0.9980908
```

Cramer-von-Mises > detectChangePointBatch(datos,"Cramer-von-Mises",alpha = 0.05)

```
$changePoint
[1] 150

$changeDetected
[1] TRUE

$alpha
[1] 0.05

$threshold
[1] 5.92981
```

alpha: El paquete incluye una función getBatchThreshold que devuelve el umbral asociado con una elección particular de alfa y n. Esta función se llama automáticamente cada vez que se llama detectChangePointBatch, por lo que el usuario solo necesita proporcionar el valor deseado de alfa. Los valores permitidos para este argumento son 0.05, 0.01, 0.005, 0.001.

Detección secuencial

En este caso, la secuencia no tiene una longitud fija. En cambio, las observaciones se reciben y procesan secuencialmente a lo largo del tiempo. Cuando se ha recibido cada observación, se toma una decisión acerca de si se ha producido un cambio basándose únicamente en las observaciones recibidas hasta el momento. Si no se marca ningún cambio, se procesa la siguiente observación en la secuencia. La formulación secuencial permite que las secuencias que contienen múltiples puntos de cambio se manejen fácilmente; cada vez que se detecta un punto de cambio, el detector de cambios simplemente se reinicia desde la siguiente observación en la secuencia. La función de detección secuencial en el paquete 'cpm' se escribe como detectChangePoint y va ser testeada con los datos de prueba de Ford para cada tipo de estadística.

ARL0: Se determina el ARL0 que debería tener el CPM, que corresponde al número promedio de observaciones antes de que se produzca un falso positivo, asumiendo que la secuencia no sufre un cambio. Solo se permiten varios valores para ARLO, específicamente, debe de tener uno de los siguientes valores: 370, 500, 600, 700, ..., 1000, 2000, 3000, ..., 10000, 20000, ..., 50000.

startup: Es el número de obsevaciones después de las cuales comienza el monitoreo. No se marcarán puntos de cambio durante este período de inicio. Esto debe establecerse en al menos 20.

Student > detectChangePoint(datos,"Student",ARL0 = 500, startup = 300)

```
$changePoint
[1] 150

$detectionTime
[1] 300

$changeDetected
[1] TRUE
```

Bartlett> detectChangePoint(datos,"Bartlett",ARL0 = 500, startup = 300)

```
$changePoint
[1] 2

$detectionTime
[1] 300

$changeDetected
[1] TRUE
```

GLR> detectChangePoint(datos,"GLR",ARL0 = 500, startup = 300)

```
$changePoint
[1] 2

$detectionTime
[1] 300

$changeDetected
[1] TRUE
```

Exponential> detectChangePoint(datos,"Exponential",ARL0 = 500, startup = 300)

```
$changePoint
[1] 150

$detectionTime
[1] 300

$changeDetected
[1] TRUE
```

FET> detectChangePoint(datos,"FET",ARL0 = 500, startup = 300, lambda = 0.3)

```
$changePoint
[1] 0

$detectionTime
[1] 0

$changeDetected
[1] FALSE
```

Mann-Whitney> detectChangePoint(datos,"Mann-Whitney",ARL0 = 500, startup = 300)

```
$changePoint
[1] 150

$detectionTime
[1] 300

$changeDetected
[1] TRUE
```

Mood> detectChangePoint(datos,"Mood",ARL0 = 500, startup = 300)

```
$changePoint
[1] 242

$detectionTime
[1] 300

$changeDetected
[1] TRUE
```

Lepage> detectChangePoint(datos,"Lepage",ARL0 = 500, startup = 300)

```
$changePoint
[1] 150

$detectionTime
[1] 300

$changeDetected
[1] TRUE
```

Kolmogorov-Smirnov> detectChangePoint(datos,"Kolmogorov-Smirnov",ARL0 = 500, startup = 300)

```
$changePoint
[1] 60

$detectionTime
[1] 300

$changeDetected
[1] TRUE
```

Cramer-von-Mises> detectChangePoint(datos,"Cramer-von-Mises",ARL0 = 500, startup = 300)

```
$changePoint
[1] 150

$detectionTime
[1] 300

$changeDetected
[1] TRUE
```

processStream: detección de múltiples cambios de punto en una secuencia

Esta función se utiliza para detectar múltiples puntos de cambio en una secuencia de observaciones utilizando el marco del modelo de punto de cambio (CPM) para la detección de cambios secuencial .Las observaciones se procesan en orden, comenzando con la primera, y se toma una decisión después de cada observación si se ha producido un punto de cambio.

A diferencia de la función detectChange, processStream no termina y regresa cuando se encuentra un punto de cambio. En su lugar, un nuevo CPM se inicializa inmediatamente después del punto de cambio, y se descartan todas las observaciones anteriores. Luego continúa el monitoreo, a partir de la primera observación después del punto de cambio. Si se descubren más puntos de cambio más adelante en la secuencia, el CPM se reinicializa nuevamente después de cada uno. De esta manera, se procesará toda la secuencia de observaciones y se pueden detectar múltiples puntos de cambio.

Student > processStream(datos,"Student",ARL0=500,startup=20)

```
$changePoints
 [1]  14  21  28  42  49  63  71  83 105 119 133 150 206 212 242 276 279 293

$detectionTimes
 [1]  20  34  41  48  62  69  83  91 108 125 139 153 209 236 247 279 296 299
```

Bartlett> processStream(datos,"Bartlett",ARL0=500,startup=50)

```
$changePoints
 [1]   2   4   6  49  51  53  55  98 100 102 104 152 154 156 206 208 210 212 263

$detectionTimes
 [1]  50  52  54  56  99 101 103 105 148 150 152 154 202 204 208 256 258 260 265
```

GLR> processStream(datos,"GLR",ARL0=500,startup=80)

```
$changePoints
 [1]   2   4   6  42  44  46  48  84  86  88  90  92  94  96  98 100 102 104 150 152 154 156 235

$detectionTimes
 [1]  80  82  84  86 122 124 126 128 164 166 168 170 172 174 176 178 180 182 184 230 232 234 240
```

Exponential> processStream(datos,"Exponential",ARL0 = 500, startup = 20)

```
$changePoints
[1] 150

$detectionTimes
[1] 166
```

FET> processStream(datos,"FET",ARL0=500,startup=80,lambda=0.3)

```
$changePoints
numeric(0)

$detectionTimes
numeric(0)
```

Mann-Whitney> processStream(datos,"Mann-Whitney",ARL0=500,startup=40)

```
$changePoints
 [1]  14  49  56  84  98 133 150 163 171 206 242 262

$detectionTimes
 [1]  40  54  89  96 124 138 173 205 203 211 246 282
```

Mood> processStream(datos,"Mood",ARL0=500,startup=20)

```
$changePoints
 [1]   7  14  35  42  63  81  99 133 150 177 185 198 241 249 256 263 292

$detectionTimes
 [1]  20  47  37  75  65  83 101 135 153 180 200 206 243 266 283 284 294
```

Lepage> processStream(datos,"Lepage",ARL0=500,startup=40)

```
$changePoints
 [1]  14  49  84  98 133 150 185 205 242 255 293

$detectionTimes
 [1]  40  54  89 124 138 173 205 225 245 282 295
```

Kolmogorov-Smirnov>
processStream(datos,"Kolmogorov-Smirnov",ARL0=500,startup=40)

```
$changePoints
 [1]  14  42  70  84 104 140 150 163 171 206 242 262

$detectionTimes
 [1]  40  54  82 110 124 144 180 190 203 211 246 282
```

Cramer-von-Mises> processStream(datos,"Cramer-von-Mises",ARL0=500,startup=40)

```
$changePoints
 [1]  14  49  56  84 104 140 150 163 171 206 242 262

$detectionTimes
 [1]  40  54  89  96 124 144 180 193 203 211 246 282
```

3.6. Repositorio not

Narrowest-Over-Threshold Change-Point Detection

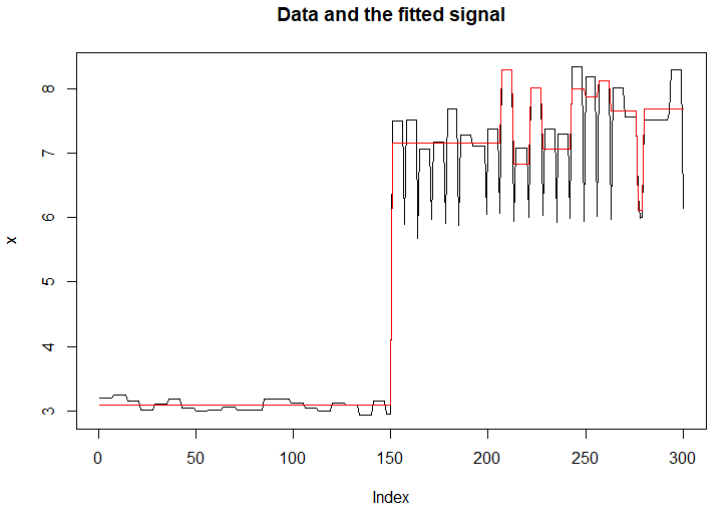
La función Narrowest-Over-Threshold, o umbral más estrecho en castellano, identifica las ubicaciones potenciales de los puntos de cambio en los datos siguiendo el modelo de "señal determinista + ruido"(ver [10]) en varios escenarios diferentes. El objeto devuelto por esta función se puede pasar a la función 'features', que encuentra la estimación final de los puntos de cambio en función del criterio de detención elegido.

cpt Indica las ubicaciones de los puntos de cambio.

contrast este parámetro determina el tipo de función contraste utilizada en el algoritmo NOT. A contiunuación, veremos con los datos de prueba de Ford del funcionamiento del algoritmo con los diferentes contrastes.

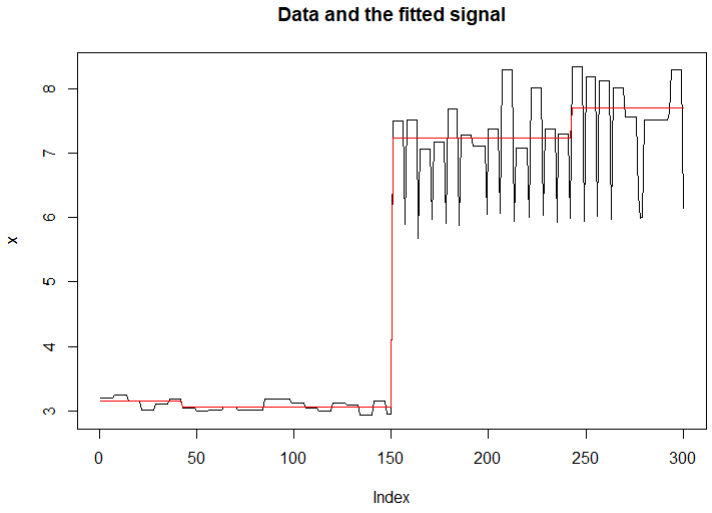
- *pcwsConstMean* Señal constante por partes con ruido Gaussiano y desviación estándar constante.

```
> not <- not(datos_ts, contrast = "pcwsConstMean")
> features(not)$cpt
[1] 150 206 212 242 248 276 279
> plot(not)
```



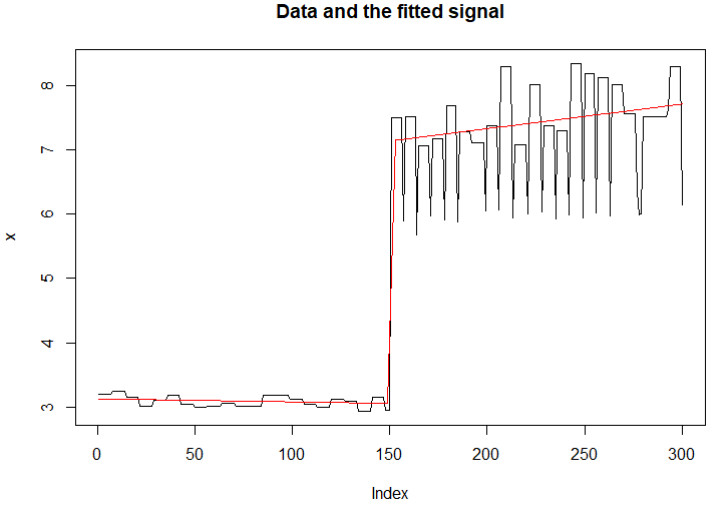
- *pcwsConstMeanHT* Media constante por partes con un ruido de cola pesada y una desviación estándar constante.

```
> not1 <- not(datos_ts, contrast = "pcwsConstMeanHT")
> features(not1)$cpt
[1]  42 150 242
> plot(not1)
```



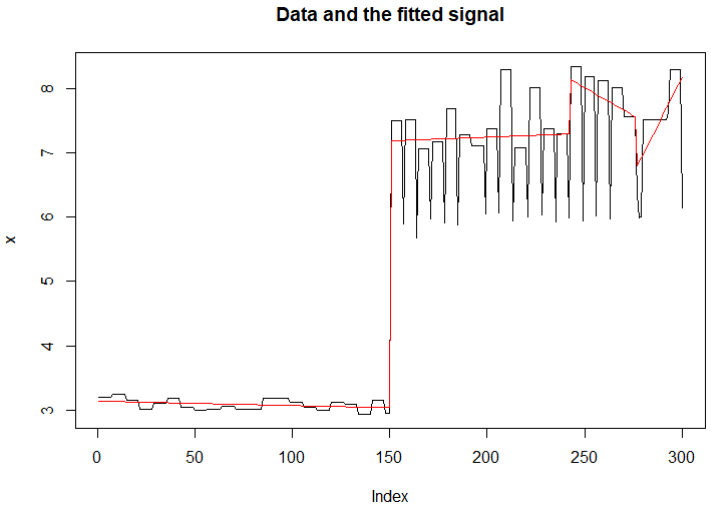
- *pcwsLinContMean* Señal continua lineal por partes con ruido Gaussiano y desviación estándar constante.

```
> not2 <- not(datos_ts, contrast = "pcwsLinContMean")
> features(not2)$cpt
[1] 149 153
> plot(not2)
```



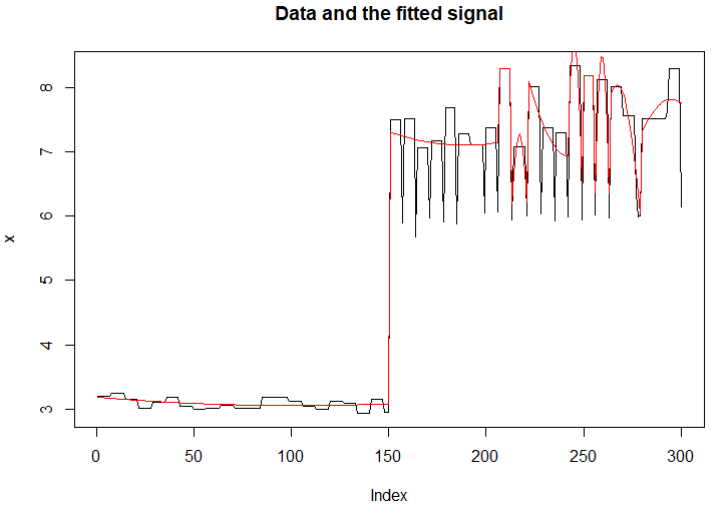
- *pcwsLinMean* Señal lineal por partes con ruido Gaussiano y desviación estándar constante.

```
> not3 <- not(datos_ts, contrast = "pcwsLinMean")
> features(not3)$cpt
[1] 150 242 276
> plot(not3)
```



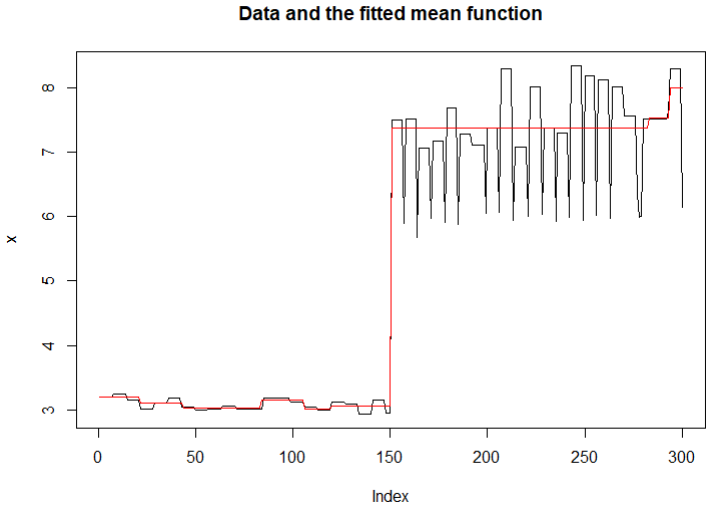
- *pcwsQuadMean* Señal cuadrática por partes con ruido Gaussiano y desviación estándar constante.

```
> not4 <- not(datos_ts, contrast = "pcwsQuadMean")
> features(not4)$cpt
[1] 150 206 212 221 242 249 255 263 279
> plot(not4)
```



- *pcwsConstMeanVar* Señal constante por partes y desviación estándar constante por partes del ruido Gaussiano.

```
> not5 <- not(datos_ts, contrast = "pcwsConstMeanVar")
> features(not5)$cpt
[1]  21  43  83 105 119 150 282 293
> plot(not5)
```



4. Resultados de cada algoritmo de punto de cambio para cada patología

4.1. Repositorio trend

Pettitt's Test

Patología	Punto estimado	p-valor	Tiempo de computo
P1. Oscilación en el cilindro clampa	686	<2.2e-16	0.02 seg
P2. Tornillo pasado en cilindro abatible fresadora	143	<2.2e-16	0.02 seg
P3. Ferricha en el cilindro clampa	803	<2.2e-16	0.01 seg
P4. Agarrotamiento pinza neumática	180	<2.2e-16	0.01 seg
P5. Agarrotamiento valvula de pinza neumática	73	= 1.043e-10	0.02 seg
P6. Fallo en el cilindro clampa	1326	<2.2e-16	0.02 seg
P7. Picos de tiempo en la pinza neumática	155	= 0.0003828	0.01 seg
P8. Fuga de aire en pinza neumática	140	= 2.916e-8	0.02 seg
P9. Picos de tiempo en la atornilladora	19568	<2.2e-16	1 seg
P10. Fallo en el cilindro no previsto	359	= 3.963e-13	0.02 seg

Buishand Range Test

Patología	Punto estimado	p-valor	Tiempo de computo
P1. Oscilación en el cilindro clampa	686	= 0.0212	33.48 seg
P2. Tornillo pasado en cilindro abatible fresadora	143	<2.2e-16	2.26 seg
P3. Ferricha en el cilindro clampa	849	<2.2e-16	55.72 seg
P4. Agarrotamiento pinza neumática	197	<2.2e-16	3.69 seg
P5. Agarrotamiento valvula de pinza neumática	73	<2.2e-16	0.38 seg
P6. Fallo en el cilindro clampa	1451	<2.2e-16	67.39 seg
P7. Picos de tiempo en la pinza neumática	61	= 0.00075	0.61 seg
P8. Fuga de aire en pinza neumática	262	= 2.916e-8	3.67 seg
P9. Picos de tiempo en la atornilladora	NULL	NULL	NULL
P10. Fallo en el cilindro no previsto	317	<2.2e-16	10.20 seg

Buishand U Test

Patología	Punto estimado	p-valor	Tiempo de computo
P1. Oscilación en el cilindro clampa	686	= 0.1082	33.69 seg
P2. Tornillo pasado en cilindro abatible fresadora	143	<2.2e-16	2.58 seg
P3. Ferricha en el cilindro clampa	849	<2.2e-16	57.67 seg
P4. Agarrotamiento pinza neumática	197	<2.2e-16	4.15 seg
P5. Agarrotamiento valvula de pinza neumática	73	<2.2e-16	0.38 seg
P6. Fallo en el cilindro clampa	1451	<2.2e-16	68.09 seg
P7. Picos de tiempo en la pinza neumática	61	= 0.05535	0.61 seg
P8. Fuga de aire en pinza neumática	262	= 2.2e-16	3.68 seg
P9. Picos de tiempo en la atornilladora	NULL	NULL	NULL
P10. Fallo en el cilindro no previsto	317	<2.2e-16	10.25 seg

Standard Normal Homogeneity Test

Patología	Punto estimado	p-valor	Tiempo de computo
P1. Oscilación en el cilindro clampa	88	= 0.1762	34.92 seg
P2. Tornillo pasado en cilindro abatible fresadora	143	<2.2e-16	2.58 seg
P3. Ferricha en el cilindro clampa	849	<2.2e-16	57.67 seg
P4. Agarrotamiento pinza neumática	197	<2.2e-16	4.15 seg
P5. Agarrotamiento valvula de pinza neumática	89	<2.2e-16	0.50 seg
P6. Fallo en el cilindro clampa	1712	<2.2e-16	69.54 seg
P7. Picos de tiempo en la pinza neumática	61	= 0.0546	0.76 seg
P8. Fuga de aire en pinza neumática	393	<2.2e-16	4.09 seg
P9. Picos de tiempo en la atornilladora	NULL	NULL	NULL
P10. Fallo en el cilindro no previsto	317	<2.2e-16	10.97 seg

En el caso que nos atañe, la simulación de la línea de soldadura de Ford Valencia se realiza basándonos en una maquina de estados, en la que sólo podemos encontrarnos en uno de los siguientes, Trabajo, Bloqueo y Espera, (ver Figura 24). En la simulación el bucle se actualiza cada 0,001 segundos. Se supone que la primera estación siempre tiene una pieza para trabajar, con lo cual nunca estará en bloqueo, además todas las piezas terminadas en la última estación son retiradas para que ésta nunca esté en estado de espera. La simulación empieza con todas las estaciones intermedias en estado de bloqueo.

El tiempo de ciclo técnico (TcT) de cada estación de trabajo (WorkStation) es el tiempo máximo de cada estación de soldadura que opera en paralelo, es decir, al trabajar las estaciones de soldadura en paralelo, el tiempo máximo de toda la estación de trabajo será el tiempo que tarda en procesar la pieza la estación de soldadura mas lenta. El tiempo de transferencia, es decir, el tiempo que se tarda en transferir una pieza de una estación de soldadura a otra, se añade a la duración del ciclo (éste se mide en 12 segundos). El tiempo de ciclo de cada aparato se calcula a través de la siguiente fórmula:

$$W = \sum_{1}^{k} T_{TC}^{s} = (\sum_{1}^{k}(\sum_{1}^{j} r_M * T_{T_{M,s}}^{M}) + \xi) + \varphi \tag{17}$$

Donde r_M son las repeticiones de cada mini-término m, ξ es el offset y φ es el tiempo de transporte. La simulación es ejecutada durante 50 horas y la media y varianza de las piezas producidas (JPH) en cada hora se computa y posteriormente se reinicia la cuenta. El resultado es el cálculo de la media y la varianza medidos en JPH $(51, 1{,}05)$. Vemos tras éste cálculo que el valor medio es igual que el ERR definido en la Figura 2.

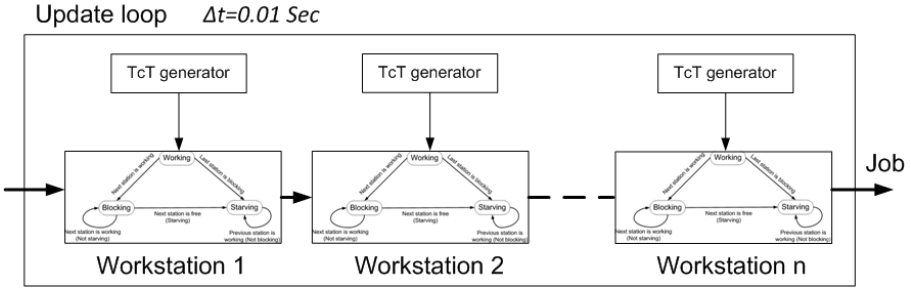


Figura 24: *Estado de la simulación de las máquinas en una estación. [5].*

5. Objetivo del proyecto

Este trabajo final de grado tiene como objetivo la creación de una aplicación informática, concretamente un cuadro de mando, que suministre información por un lado a la alta dirección mostrando niveles de rendimiento de los elementos de las estaciones de soldadura además de detección de cuellos de botella en la linea, y por otro lado al personal de mantenimiento con una visualización en tiempo real de las mediciones de los tiempos empleados para los procesos de soldadura unido a un sistema de alertas de anomalías.

5.1. Definición Cuadro de Mando

Se define el término Cuadro de Mando Integral (CMI), en ingles Balanced Scorcard – (BSC), según [16], como la *"herramienta revolucionaria para movilizar a la gente hacia el pleno cumplimiento de la misión a través de canalizar las energías, habilidades y conocimientos específicos de la gente en la organización hacia el logro de metas estratégicas de largo plazo. Permite tanto guiar el desempeño actual como apuntar al desempeño futuro. Usa medidas en cuatro categorías -desempeño financiero, conocimiento del cliente, procesos internos de negocios y, aprendizaje y crecimiento- para alinear iniciativas individuales, organizacionales y transdepartamentales e identifica procesos enteramente nuevos para cumplir con objetivos del cliente y accionistas. El BSC es un robusto sistema de aprendizaje para probar, obtener retroalimentación y actualizar la estrategia de la organización. Provee el sistema gerencial para que las compañías inviertan en el largo plazo -en clientes, empleados, desarrollo de nuevos productos y sistemas- más que en gerenciar la última línea para bombear utilidades de corto plazo. Cambia la manera en que se mide y maneja un negocio"*.

El CMI es por tanto un sistema de gestión estratégica de la empresa.

El concepto de CMI, es un concepto demasiado genérico y sobretodo complejo que excede con mucho el objetivo de este proyecto; por tanto introducimos el concepto de *Dashboard* que refleja algunas características teóricas del CMI. De forma genérica, un dashboard engloba a varias herramientas que muestran información relevante para la empresa a través de una serie de indicadores de rendimiento, también denominados KPI's (key performance indicators). Cabe señalar que un dashboard puede no ser balanceado, término que evoca al "Balanced"Scorecard, es decir un balance entre indicadores que visualicen en forma transversal la organización o empresa y que quizás para un dashboard solo puede buscar y dirigir su mirada a un conjunto focalizado y parcial de indicadores [17].

Los CMI's, o en caso caso mas concreto los Dashboards se pueden subdividir en Cuadros de Mando Estratégicos (CME), si contienen indicadores estratégicos y Cuadros de Mando Operativos (CMO), si los indicadores son operativos, siendo estos últimos el objetivo de este trabajo.

5.2. KPI's para Ford Valencia

Para el caso particular de la planta de carrocerías de Ford Valencia, se han escogido una serie de indicadores de rendimiento que serán utilizados por distinto personal dentro de la planta; por un lado mostraremos cuatro tipos de KPI's para la gerencia de la empresa, que analizará los tiempos de ciclo técnico (TcT) de las estaciones de trabajo, detectando cuellos de botella en la linea; y por el otro lado, para el personal de mantenimiento, mostraremos un KPI en tiempo real con los tiempos medios de cada estación de soldadura, monitorizando las desviaciones y alertando de ellas. De manera resumida, los KPI's diseñados son los siguientes:

- KPI's para el personal de Gerencia.

  - Indicador de Tiempos donde se muestran las medidas de los tiempos de parada, bloqueo y TcT de cada estación de trabajo junto con los datos de los posibles cuellos de botella detectados, ademas de un sub-KPI donde se muestra que estación de soldadura (robot) es el causante del cuello de botella y para que mini-término.

  - Indicador de la producción mensual de un modelo en concreto diferenciado por días y calculando la media del mes.

  - Indicador del TcT y Offset de cada estación de trabajo.

  - Indicador del tiempo ocioso (Idle Time) de cada estación de trabajo.

- KPI's para el personal de mantenimiento.

  - Indicador de monitorización en tiempo real de la linea de soldadura detectando anomalías en el funcionamiento de los mini-términos en los que se divide cada estación de soldadura dentro de cada estación de trabajo. Estas anomalías muestran alertas en caso de rebase de umbrales.

Estos KPI's serán analizados en profundidad en la sección de explicación de la aplicación propuesta.

6. Aplicación Cuadro de Mando

6.1. Concepto general

Como hemos comentado anteriormente, la aplicación informática desarrollada está planteada para su uso por parte de gerencia, para el control de los tiempos de TcT, parada, bloqueo, offset e idle time así como la medición del nivel de producción mensual; y para el departamento de mantenimiento para la medición en tiempo real de los tiempos de ejecución de las estaciones de soldadura.

Para este propósito, se podrían haber utilizado diversos programas existentes en el mercado tanto de software libre como propietario, tales como Cyfe [18] o ClicData [19] entre otras. Pero dada la característica de los datos a medir (datos provenientes de mediciones en tiempo real), así como la forma de almacenarla (base de datos o estructura dinámica), se optó por desarrollarla desde cero programando los KPI's directamente en el lenguaje de programación y mostrando los datos desde código.

6.2. Elección de lenguajes y tecnologías

Debido a la complejidad en recrear un sistema de indicadores gráficos y alertas partiendo de la nada, se decidió utilizar un lenguaje de programación robusto y orientado a los objetos que se usarán en la aplicación y que mostrarán al usuario los diferentes KPI's.

El lenguaje escogido ha sido C++ por su versatilidad y potencial a la hora de gestionar los diferentes objetos y eventos creados para la aplicación. C++ es una extensión del lenguaje C (que forma parte de los llamados lenguajes estructurados), con el añadido de la orientación a objetos (el término C++ proviene del concepto "incremento de C"). Es un lenguaje multiparadigma.

Junto a este lenguaje de programación se han utilizado también librerías disponibles en el framework de Qt en su versión 5, ya que contiene herramientas gráficas que facilitan la creación de formularios gráficos.

Para la creación de los formularios del cuadro de mando se ha utilizado Qt Creator, un IDE (Entorno de Desarrollo Integrado) multiplataforma para desarrolladores de Qt. En la Figura 25 se muestra la pantalla de inicio del IDE Qt Creator [20].

Dentro de los formularios del cuadro de mando que interactúan con el usuario, se ha utilizado un Widget de Qt C++ denominado QCustomPlot que da la posibilidad de programar graficos customizados practicamente con cualquier formato [**?**].

Para la aplicación de escritorio se ha utilizado un generador de instaladores denominado Install Creator 2, cuyo cometido ha sido la creacion del instalador para la plataforma Windows [**?**].

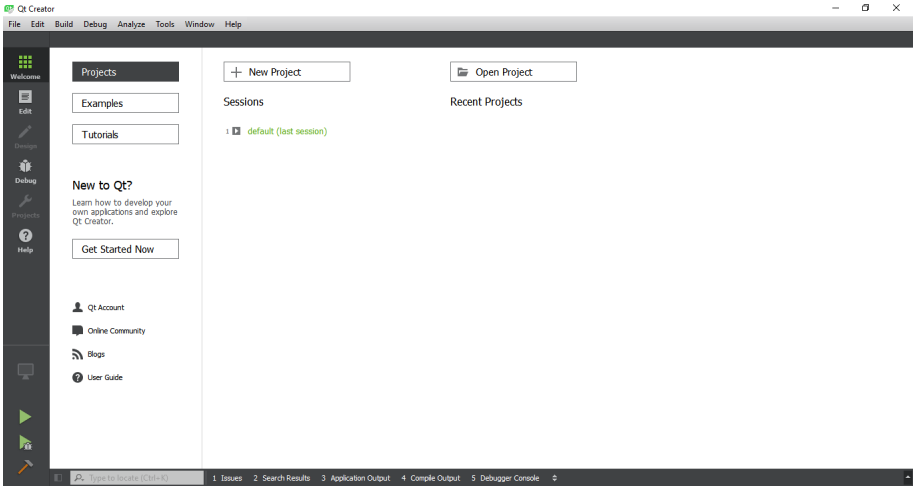


Figura 25: *Pantalla principal del IDE Qt Creator.*

6.3. Captura de datos

Los datos a procesar provienen de los sensores de cada mini-término de cada robot o estación de soldadura, por tanto debemos proveer de un sistema de captación de datos que nos permita poder procesarlos posteriormente con facilidad. Se está trabajando desde Ford Valencia en la creación de una gran base de datos que albergue los datos que registran los sensores de cada máquina, aunque a día de finalización de este proyecto todavía no se había generado la base de datos; los datos se han tenido que guardar temporalmente en estructuras creadas para tal fin en el mismo código. Estas estructuras se corresponden con matrices (Arrays) creadas mediante código y que son rellenadas con lecturas en streaming desde ficheros planos creados para contener los datos que envían los sensores. Estos ficheros se corresponden con los que se crearon para la tesis del Dr. Magraner [5] denominados Estacion_modelos.txt y Produccion.txt.

Para la medición de los tiempos de TcT, bloqueo y espera de cada estación de trabajo y posterior detección de cuellos de botella, se ha creado un algoritmo en C++ que una vez ejecutado es capaz de obtener la media y varianza de cada mini-término basándose en los datos que envían los sensores de cada estación de soldadura (de momento esta medición es experimental ya que no está establecido el sistema de sensores en la planta). Partiendo de este algoritmo se generan dos ficheros llamados tiempos.txt y rendimiento.txt que son recogidos por la aplicación para mostrar en forma de gráfica el comportamiento en tiempo real de las estaciones de trabajo.

6.4. Funcionamiento de la aplicación

Aunque originariamente la aplicación fue diseñada para ser un sistema de escritorio, se ha visto que debido a la actual demanda de dispositivos móviles para su uso en entornos de producción, era necesario dotar a la aplicación de esa movilidad que podía hacerla mucho mas versátil y facilitar al personal de gerencia y mantenimiento la visualización de los datos aun cuando no se encontraran físicamente en su lugar

de trabajo.

Aprovechando que la tecnología de desarrollo elegida había sido el IDE Qt Creator, que recordemos es multiplataforma, se pudo adaptar la aplicación a la plataforma mas extendida hoy en día en cuanto a dispositivos móviles; hablamos por supuesto de Android, cuya cuota de mercado en cuanto al uso en España se muestra en la Figura 26.

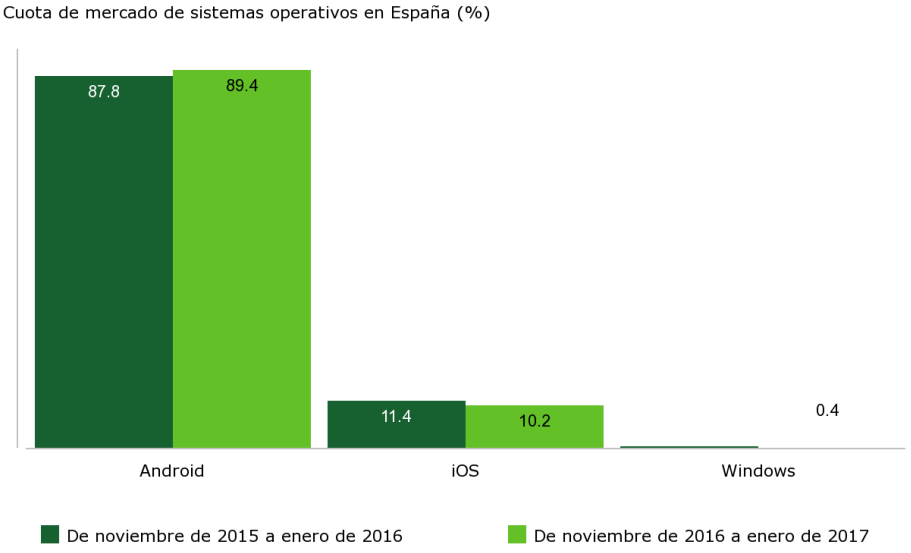


Figura 26: *Cuota de mercado de sistemas operativos en España (%) [21].*

La aplicación en cuestión está pensada para un entorno cliente-servidor, es decir, un servidor alojado en la empresa albergará las bases de datos a las que atacará la aplicación, ademas del fichero que ejecuta el algoritmo que calcula los tiempos de TcT, parada y bloqueo de las estaciones y el rendimiento (media y varianza) de las mismas (se explica a continuación); y varios clientes instalados en los ordenadores de los usuarios que contienen la aplicación en si.

6.5. Instalaciones en el servidor

El sistema propuesto a continuación deberá ser instalado en el servidor de la empresa en cuanto esté listo el sistema de sensorización de las estaciones de soldadura. De momento todo este proceso está integrado en el propio código mediante estructuras (arrays) y gestionado por funciones que simulan ataques a la base de datos, aunque en realidad se efectúan sobre ficheros planos guardados en la propia carpeta del proyecto. De todas formas se explica a continuación cual es el procedimiento a realizar para dejar el sistema preparado para el correcto funcionamiento de la aplicación en un futuro.

Necesitamos tener ciertas bases de datos albergadas en el servidor de manera que la aplicación pueda atacarlas.

El sistema gestor de base de datos elegido (SGBD) ha sido PostgreSQL [22], ya que es software libre y orientado a aplicaciones de escritorio. Podíamos haber elegido MySql por ser también de software libre, pero este SGBD está mas orientado a aplicaciones web y al uso con lenguajes del lado del servidor como PHP.

Las bases de datos necesarias para el correcto funcionamiento son:

- Base de datos denominada 'usuarios' que contendrá los datos de los usuarios que acceden a la aplicación (id, usuario, contraseña, rol, activo). El proyecto de momento solo tiene dos usuarios (gerente y mantenimiento).

- Base de datos denominada 'roles', que albergará los distintos roles de los usuarios de la aplicación (id, rol). El proyecto de momento solo tiene dos roles (gerente y mantenimiento).

- Base de datos denominada 'estacion_modelos' con los tiempos y repeticiones de cada robot en cada uno de sus mini-términos y offset. Las columnas contendrán los tiempos del primer mini-término y las repeticiones del segundo y tercer mini-término, ademas del offset del robot. Las filas marcarán cada robot de los que componen la workstation.

- Base de datos denominada 'produccion' con la cantidad de unidades producidas. Las columnas representarán los días del mes y las filas se irán construyendo con el identificador del año, el identificador del mes y la producción del día del mes (desde el día 1 al día 31).

- Base de datos denominada 'anios', con los años a procesar (id, anio).

- Base de datos denominada 'meses' con los meses a procesar (id, mes).

- Base de datos denominada 'rendimiento' que contendrá los datos de las medias y varianza de los 68 modelos fabricados en Ford valencia.

- Base de datos denominada 'tiempos' que contendrá los tiempos de TcT, espera y bloqueo de las 8 estaciones de trabajo para cada modelo fabricado.

La Figura 27 muestra la estructura de la base de datos en el servidor.

Junto con la base de datos PostgreSQL instalaremos la aplicación pgAdmin [23], que se encarga de administrar y gestionar las bases de datos.

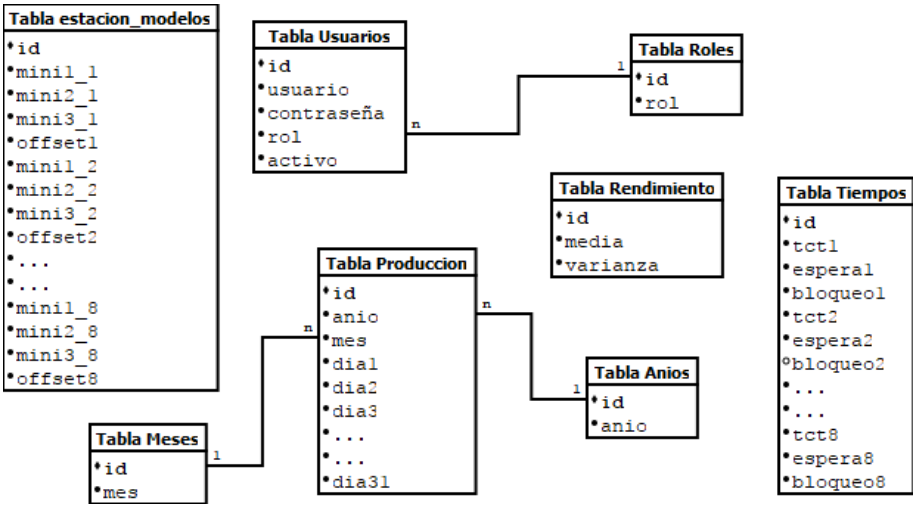


Figura 27: *Estructura de la base de datos en el servidor.*

Necesitamos instalar también, el algoritmo que genera los tiempos de TcT, espera y bloqueo de cada modelo, así como también los rendimientos (media y varianza) de cada modelo.

El sistema básicamente funciona de la siguiente forma: los datos que se van recibiendo de los robots son procesados por el algoritmo instalado en el servidor que por un lado irá rellenando la tabla 'estacion_modelos' y por otro generando los datos que se insertarán, en las tablas 'rendimiento' y 'tiempos'.

La tabla de 'produccion' se irá rellenando con los datos de las producciones diarias que se vayan generando.

Las tablas 'usuarios', 'roles', 'anios' y 'meses' son estáticas, es decir no se van actualizando conforme cambian las condiciones en fabrica, por lo tanto se rellenan al principio (antes de poner en marcha el sistema) y se insertaran nuevos registros conforme vayamos cambiando de año, mes o creando mas usuarios y roles en la aplicación.

En la Figura 28 se muestra el esquema del proceso completo.

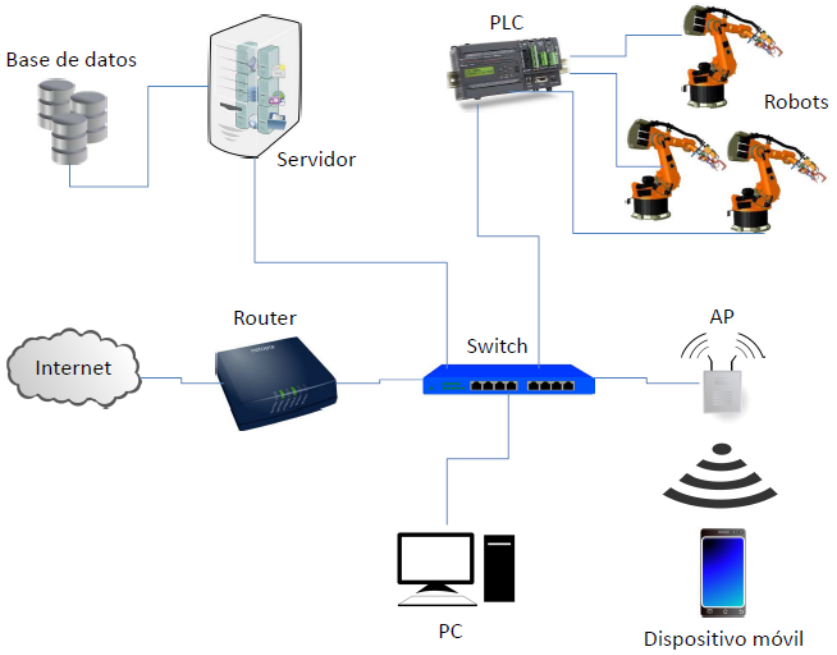


Figura 28: *Esquema del proceso completo (Fuente propia).*

6.6. Instalaciones en los clientes

La instalación de la aplicación en los clientes únicamente requiere ejecutar el instalador que viene preparado para la plataforma Windows o ejecutar el archivo .apk para Android.

Instalación en Android:

La aplicación para la plataforma Android está preparada para ser instalada tanto en una tablet como en un smartphone. Únicamente se copiará el archivo .apk al dispositivo (por ejemplo mediante un cable USB conectado con un pc que contenga la aplicación). Es importante señalar, que se debe activar la opción de *Orígenes Desconocidos* dentro del apartado de *Seguridad* en los ajustes del dispositivo móvil ya que si no nos dará un error a la hora de instalar la aplicación. En las siguientes figuras se muestran algunas pantallas de la aplicación instalada tanto en un smartphone de 5.5"como en una tablet de 10.1".

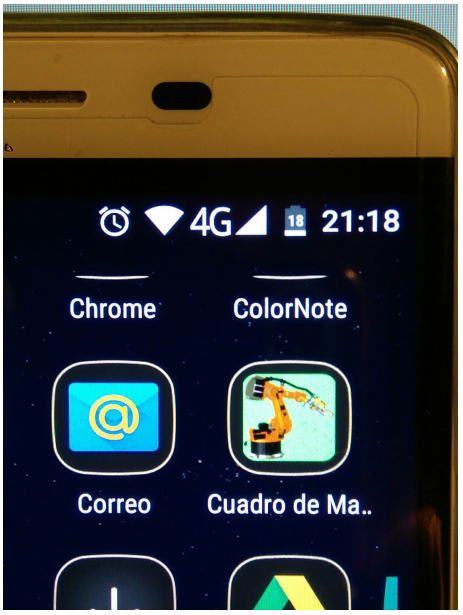


Figura 29: *Aplicación instalada en un smartphone.*

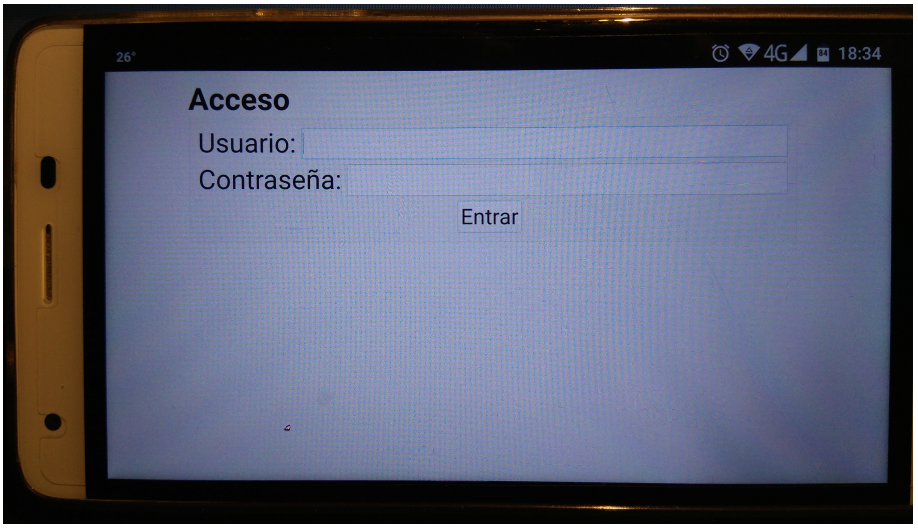


Figura 30: *Pantalla de login en smartphone.*

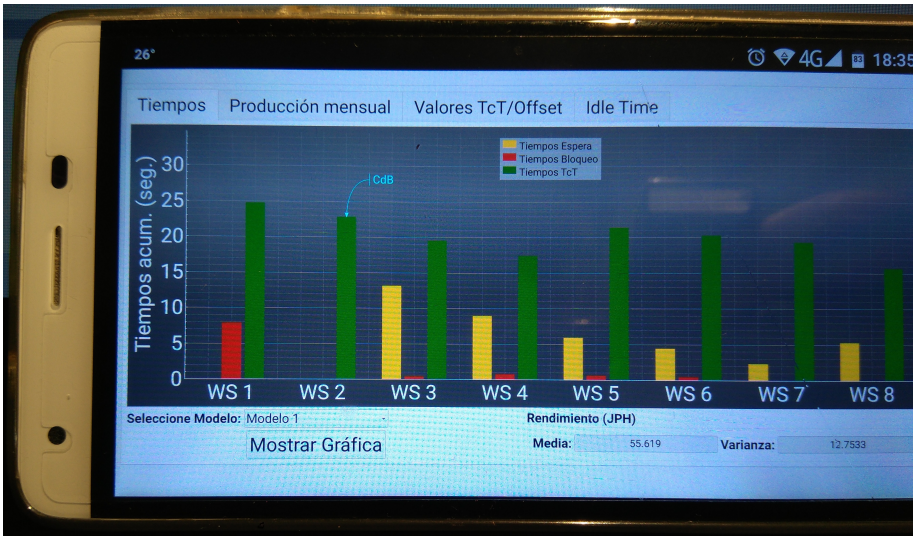


Figura 31: *Pantalla para usuarios de gerencia en smartphone (Tiempos TcT).*

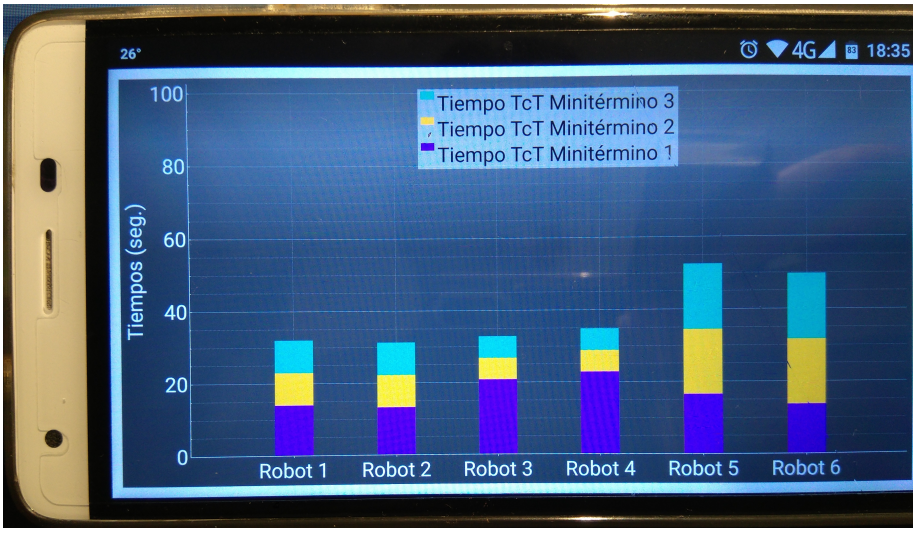


Figura 32: *Pantalla para usuarios de gerencia en smartphone (detalle cuello de botella).*

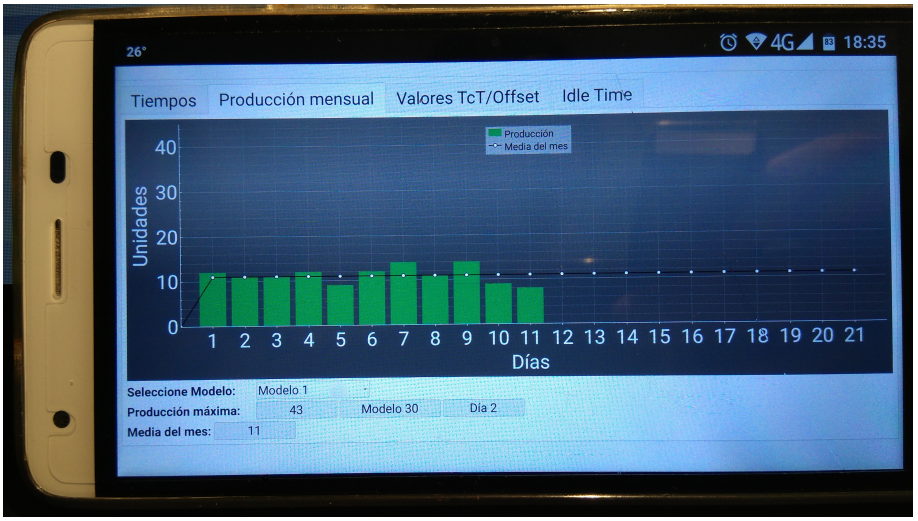


Figura 33: *Pantalla para usuarios de gerencia en smartphone (Producción mensual).*

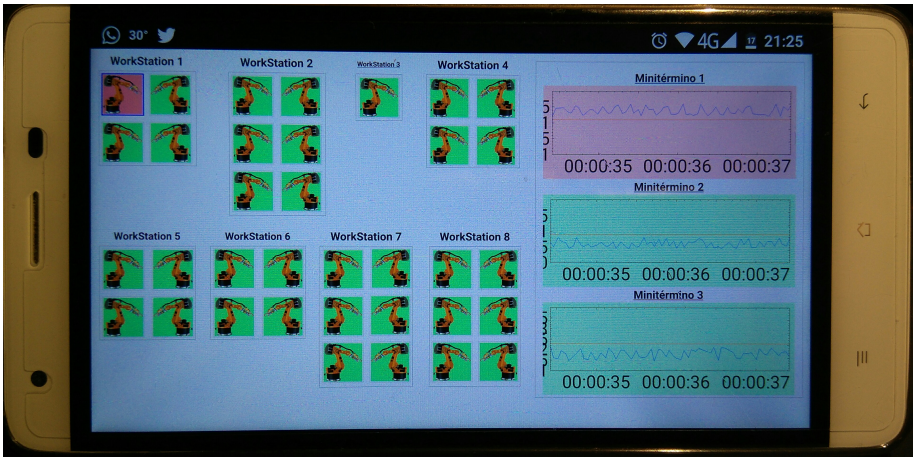


Figura 34: *Pantalla para usuarios de mantenimiento en smartphone.*

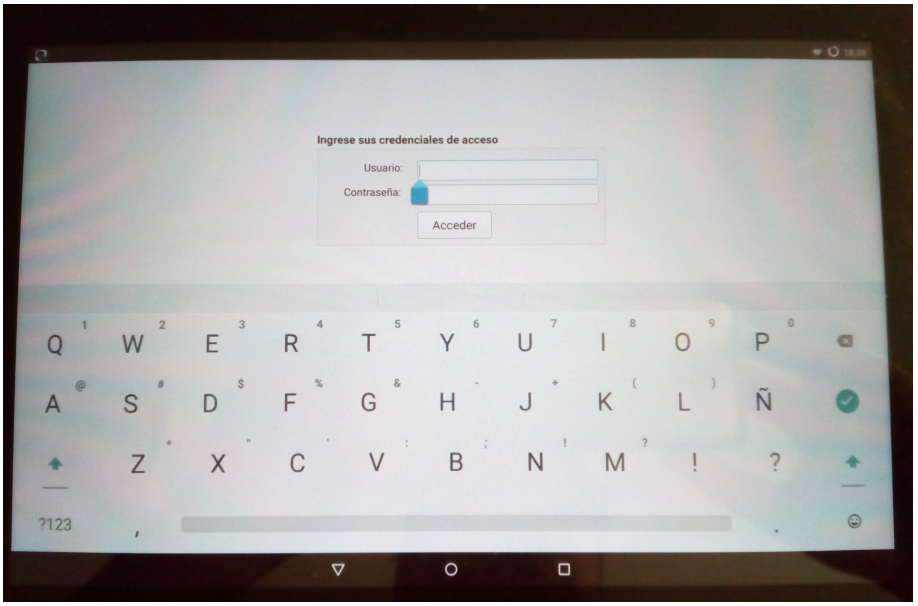


Figura 35: *Pantalla de login en tablet.*

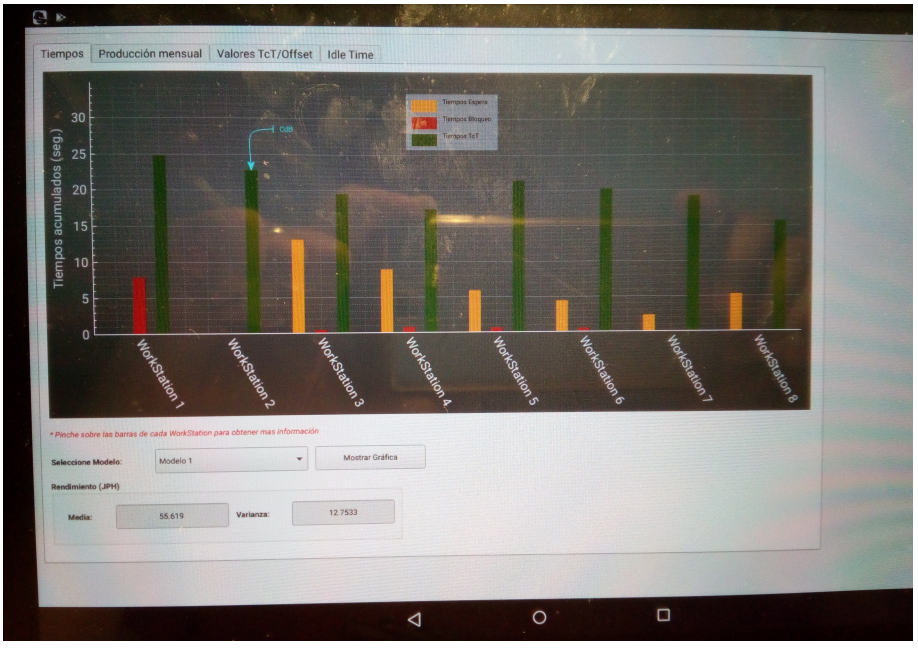


Figura 36: *Pantalla para usuarios de gerencia en tablet (Tiempos TcT).*

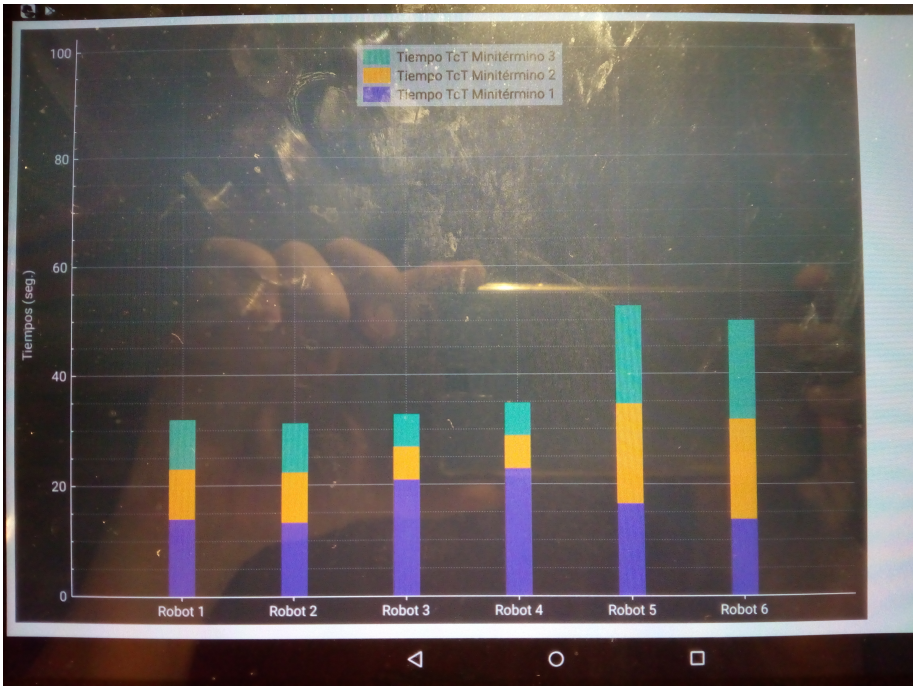


Figura 37: *Pantalla para usuarios de gerencia en tablet (detalle cuello de botella).*

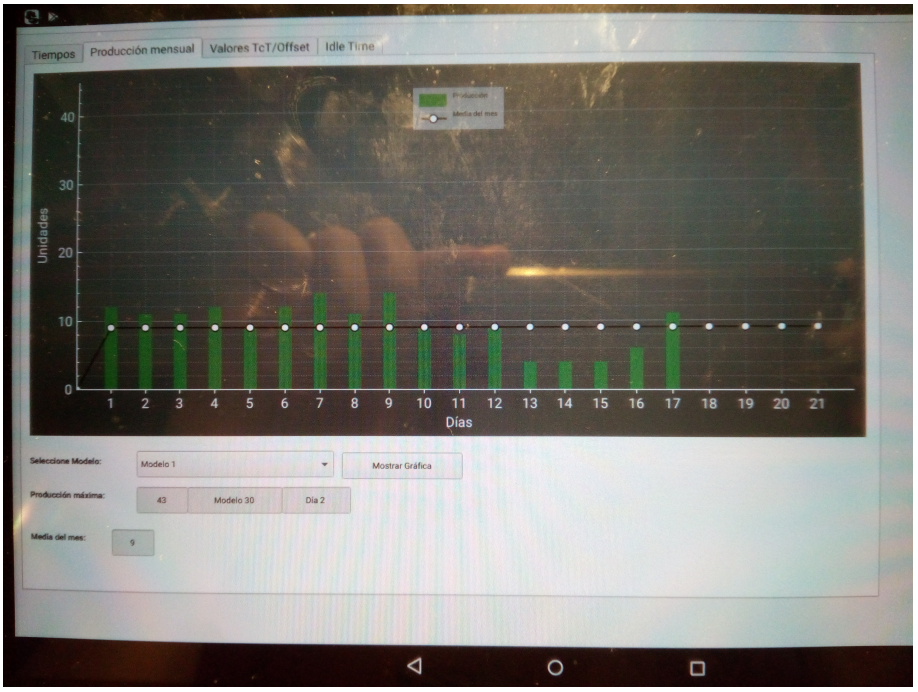


Figura 38: *Pantalla para usuarios de gerencia en tablet (Producción mensual).*

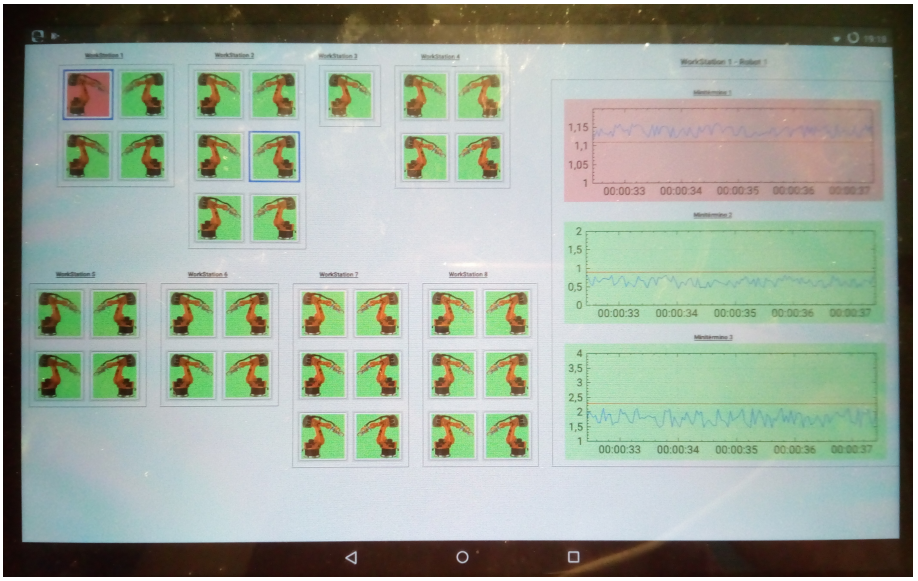


Figura 39: *Pantalla para usuarios de mantenimiento en tablet.*

Instalación en Windows:

La instalación en la plataforma Windows se realizará mediante un instalador creado para tal fin. Daremos click al archivo CuadroMandoInstall.exe desde el equipo en el cual queremos instalarla.

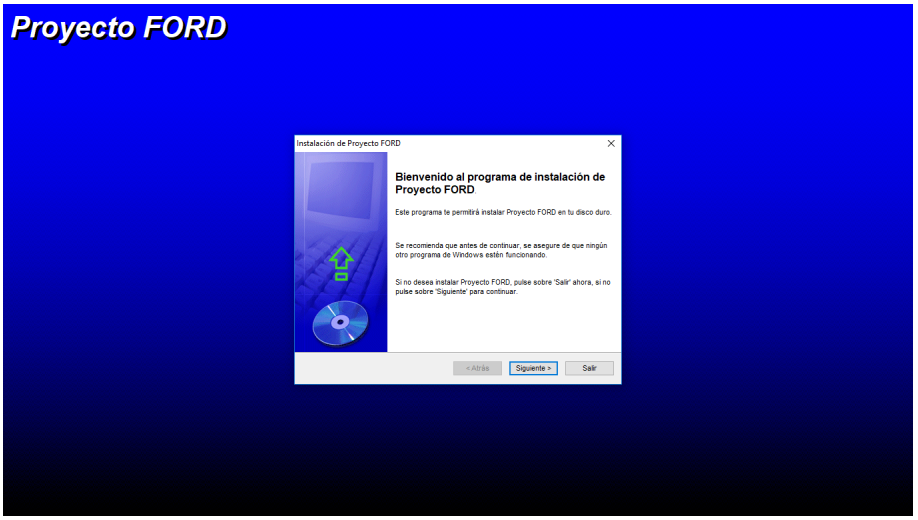


Figura 40: *Pantalla principal del instalador de la aplicación.*

Pulsaremos *Siguiente* para continuar con la instalación.

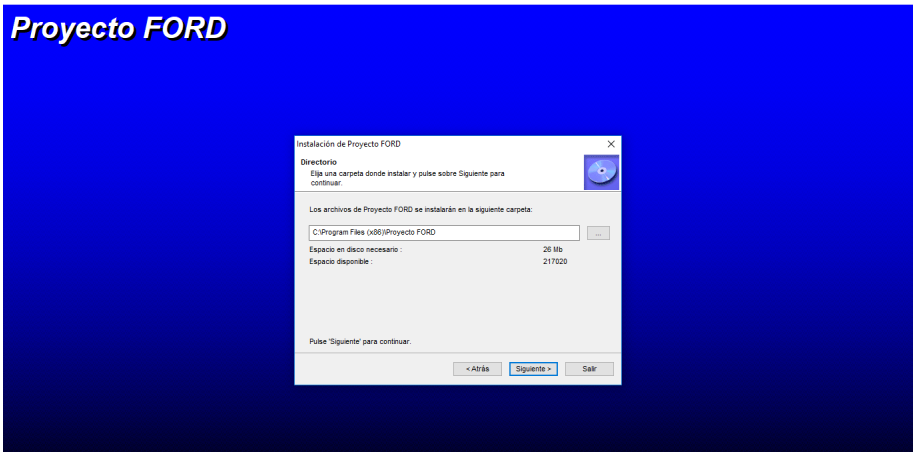


Figura 41: *Pantalla para seleccionar ubicación de la aplicación.*

Pulsaremos nuevamente *Siguiente* para continuar.

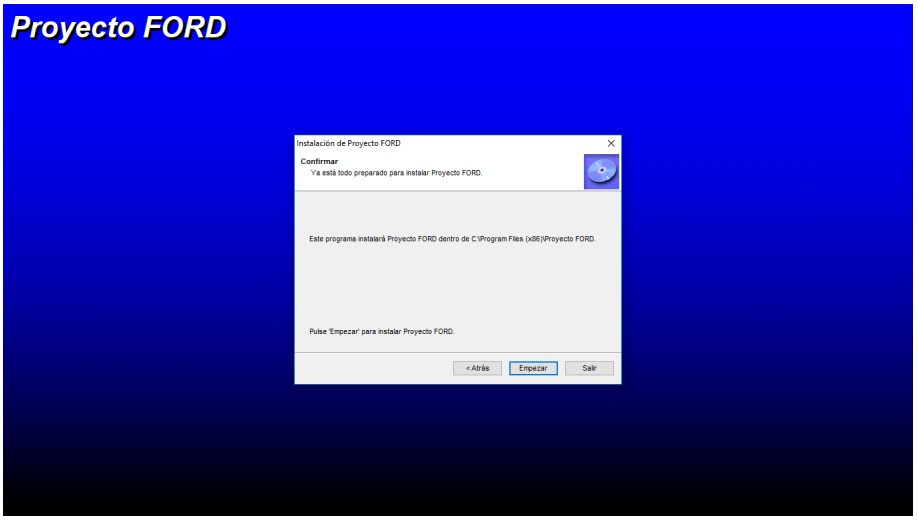


Figura 42: *Pantalla de confirmación.*

Pulsaremos sobre el botón *Empezar* para comenzar la instalación.

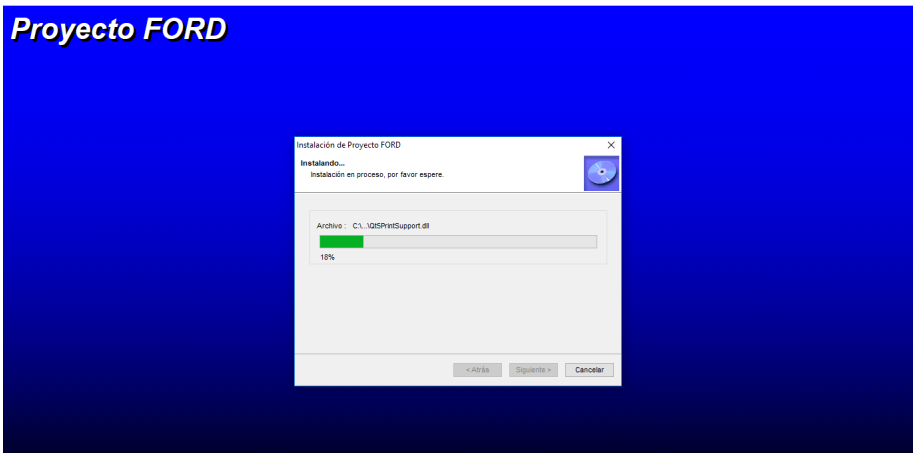


Figura 43: *Pantalla con el progreso de la instalación.*

Una vez ha concluido la instalación de la aplicación, el instalador nos muestra una pantalla indicando si ha habido algún problema o por el contrario se ha instalado correctamente.

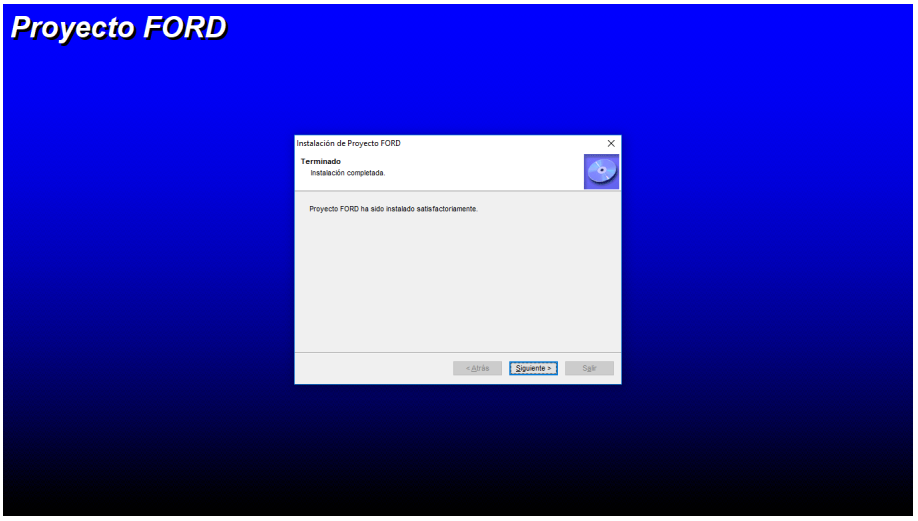


Figura 44: *Pantalla con el resultado de la instalación.*

Una vez pulsemos sobre el botón *Siguiente* la aplicación se habrá instalado correctamente y tendremos un acceso directo en el escritorio.

Figura 45: *Acceso directo de la aplicación en el escritorio.*

Una vez instalada la aplicación, haremos doble click en el icono creado en el escritorio y accederemos a la pantalla de login. He de comentar que las contraseñas en la aplicación no se guardan *"en claro"* en la base de datos, éstas se guardan después de haber aplicado una huella (hash) sobre ella, concretamente una función SHA1 que genera un código hexadecimal sobre el documento aplicado. Una vez introducimos la contraseña en la celda correspondiente del formulario de logueo, se calcula la huella en formato SHA1 sobre ella y se compara con la huella guardada en la base de datos, si coincide accedemos y si no muestra un mensaje de error. La pantalla de login se muestra a continuación:

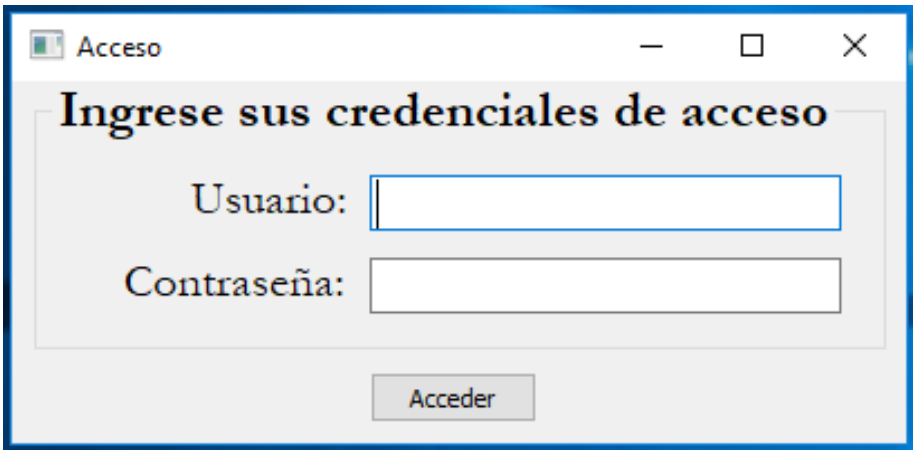


Figura 46: *Pantalla de introducción de usuario y contraseña.*

La aplicación está preparada por el momento para dos tipos de roles, el de gerencia y el de mantenimiento. Las credenciales de acceso para gerencia serán usuario: *gerente* y contraseña: *gerente*, y para mantenimiento usuario: *mantenimiento* y contraseña: *mantenimiento*.

Veamos primero la parte de gerencia.

Una vez nos hemos identificado, la aplicación nos muestra una pantalla principal con la fecha en la barra de estado en la parte inferior izquierda y sobre ella un formulario distribuido en pestañas con los datos principales de la aplicación.

En la primera pestaña, denominada *Tiempos* podemos ver los tiempos de Espera, Bloqueo y TcT de cada WorkStation que nos llegan directamente de las mediciones de los sensores de la linea. En la Figura 47 podemos ver el detalle de la primera pestaña. En la parte superior aparece la gráfica con los tiempos mencionados y en la inferior mostramos los datos del rendimiento medido en Jobs per Hour (JPH) con la media y varianza del modelo seleccionado, este modelo podemos seleccionarlo mediante un desplegable en el que se encuentran los 68 modelos fabricados por Ford Valencia. Vemos además que el sistema es capaz de detectar cuellos de botella en la linea, estos están marcados mediante un texto en cyan indicando con una flecha la estación que ha entrado en cuello de botella.

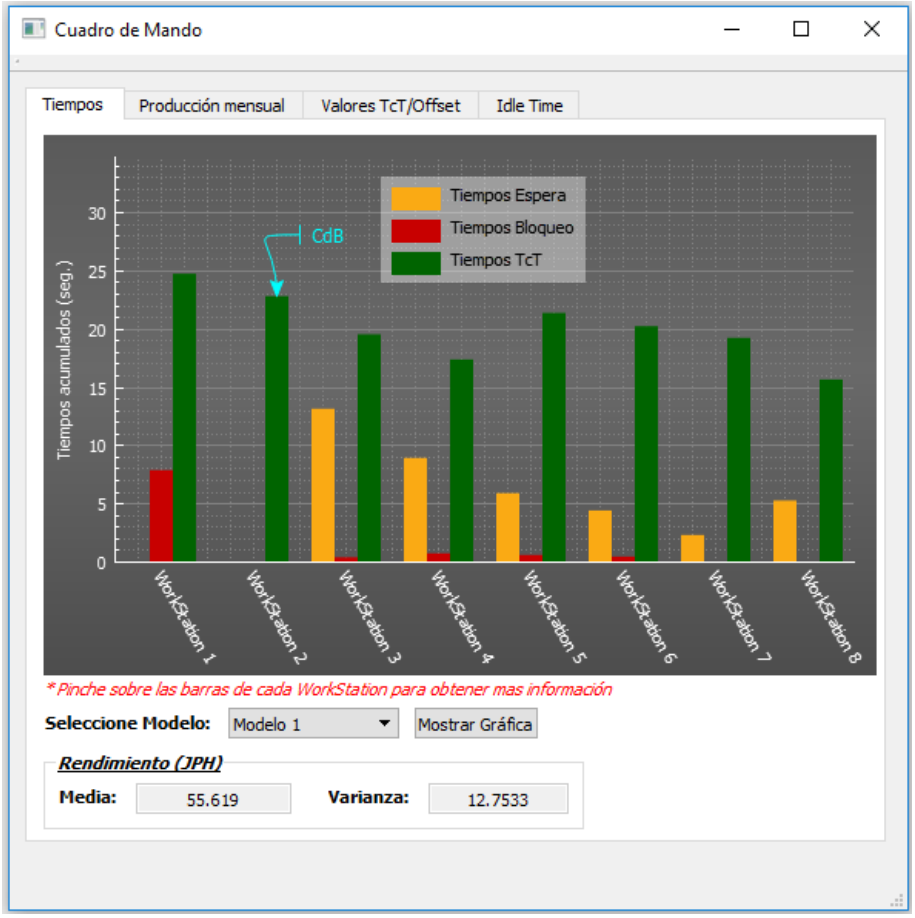


Figura 47: *Pestaña de Tiempos.*

Podemos, pinchando en la indicación del cuello de botella acceder a otra gráfica donde se nos mostrará cual es el robot dentro de la WorkStation que esta provocando el cuello de botella, indicando separadamente los tiempos de TcT de cada uno de ellos para evaluar como actuar (ver Figura 48).

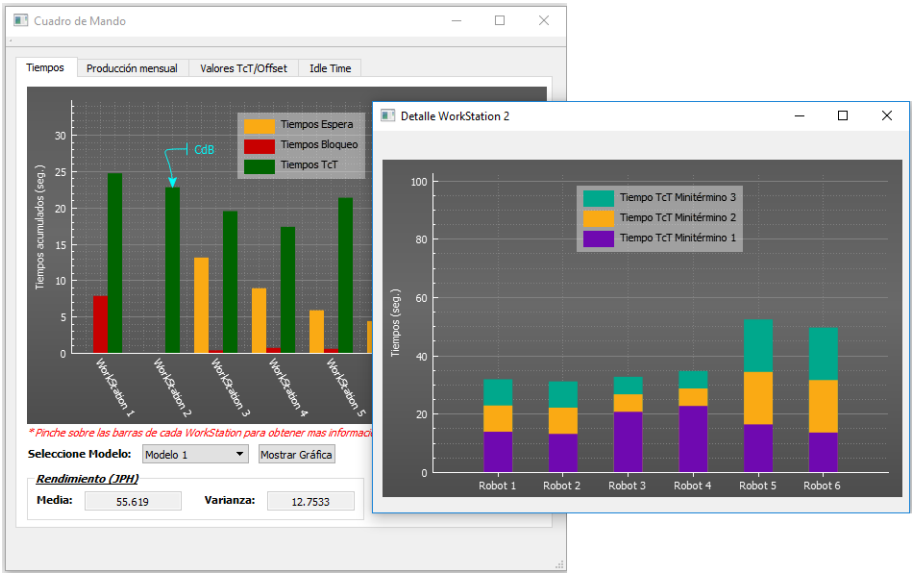


Figura 48: *Pantalla de detalle del cuello de botella (CdB).*

La siguiente pestaña que podemos ver en la aplicación es la denominada *Producción Mensual* (ver Figura 49), en la que podemos ver una simulación (recordemos que a la fecha de finalización de este trabajo fin de grado no estaba listo el sistema de sensorización de Ford Valencia) de la producción mensual en días. Cada 5 segundos se simula un día de producción según el modelo seleccionado; este también se puede elegir desde el desplegable de la parte inferior. Además de los datos anteriores, la aplicación también muestra la media del mes de los días que llevamos producidos, además va recalculándose cada vez que se genera un nuevo valor; mostramos también los datos de cual ha sido la producción máxima hasta la fecha, para qué modelo y día.

La tercera pestaña llamada *Valores TcT/Offset* (ver Figura 50) muestra los datos de Tiempo de Ciclo Técnico (TcT) y Offset (tiempo que está parada) cada estación de trabajo, es decir, el tiempo que no está trabajando con ninguna pieza. Esta gráfica nos da una visión de qué WorkStation necesita optimización, una estación con demasiado offset que realiza su trabajo demasiado deprisa o que las unidades adyacentes lo hacen demasiado despacio, con lo cual se puede optimizar el rendimiento variando la velocidad de las estaciones para disminuir ese tiempo.

La pestaña muestra también cual es el tiempo de TcT y Offset máximos que se están obteniendo, qué modelo los está generando y en qué WorkStation se están produciendo.

Además de eso, la pestaña también nos permite seleccionar el modelo a visualizar.

La última pestaña denominada *Idle Time* muestra el tiempo ocioso de cada WorkStation, es decir nos permite ver una gráfica donde aparecen los tiempos en los que cada estación de trabajo ha estado en espera, en bloqueo o en los dos estados. Hace un estudio de cada estación de trabajo para la asistencia en la toma de decisiones de rendimientos de la linea, maquinas demasiado ociosas son desaprove-

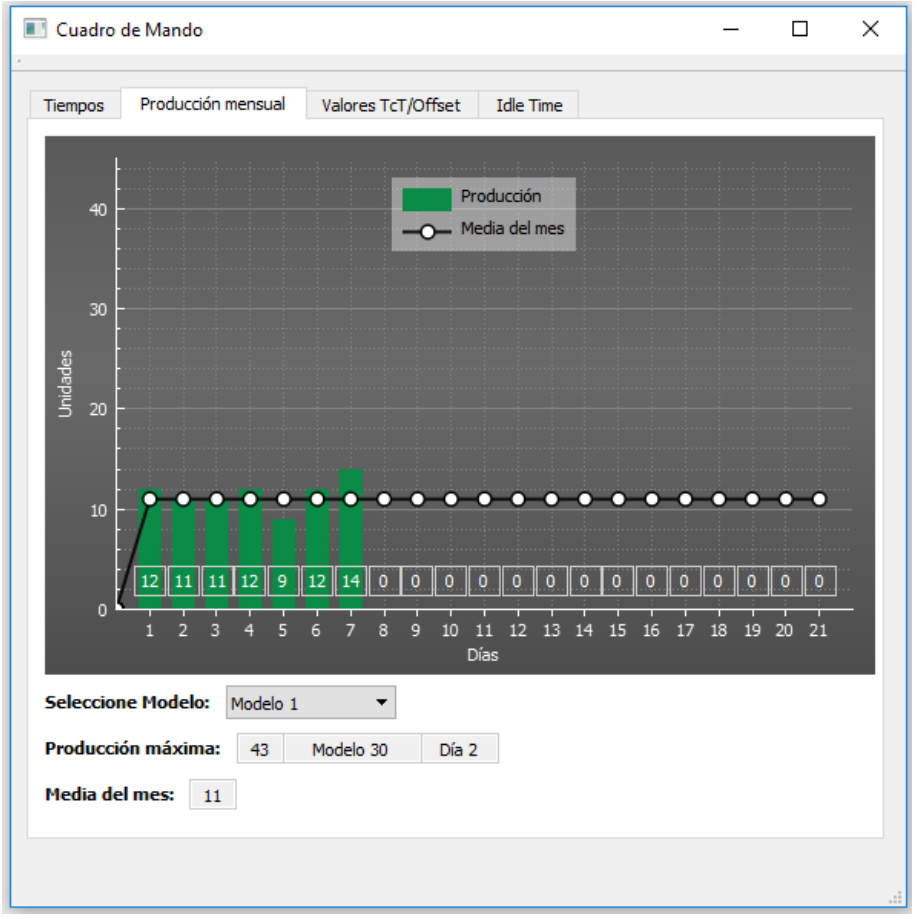


Figura 49: *Pestaña de Producción.*

chamiento de las capacidades de las mismas.

Al igual que en resto de pestañas, en esta también se puede seleccionar el modelo para ver su gráfica.

La Figura 51 muestra el detalle:

Después de analizar la parte de la aplicación destinada a la gerencia, se estudia a continuación la destinada a mantenimiento. Según podemos ver en la Figura 52, la aplicación muestra en la parte izquierda los botones, de manera simbólica, de todos los robots implicados en la linea de soldadura objeto de monitorización, agrupados por WorkStation; estas WorkStation contienen desde 1 hasta 6 robots soldadores (estaciones de soldadura). Cada vez que pulsamos en uno de ellos, se nos muestra en la parte derecha de la ventana una gráfica en tiempo real de los tiempos que se tarda en realizar cada tarea (tiempos que coinciden con los de los tres mini-términos en los que se descompone cada robot). Estos tiempos se corresponden con las tres gráficas que nos encontramos en la parte derecha.

Estos tiempos se deben ajustar, en mayor o menor medida, a las medidas experimentales obtenidas en [5]. Estas medidas experimentales concretaron unas medias

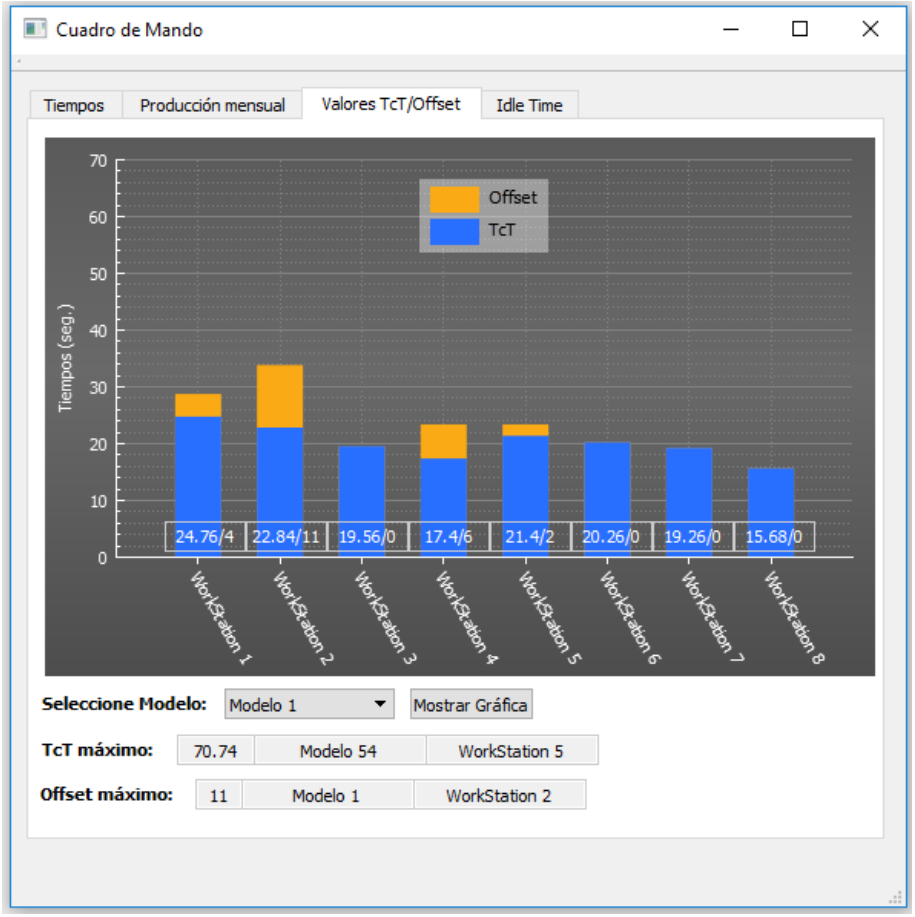


Figura 50: *Pestaña de Valores TcT/Offset.*

y varianzas según las distintas repeticiones de las tareas, con lo cual se establece un limite superior definido por la suma de la media de los tiempos obtenidos experimentalmente mas su varianza. Este limite superior es el que nos indicará si un robot en concreto está marcando tiempos por encima de lo normal para la tarea encomendada. En caso de que estos tiempos superen el umbral máximo establecido (media + varianza), tanto el color del botón, como el de la gráfica del mini-término afectado cambiarán a rojo, indicando la anomalía (ver Figura 53).

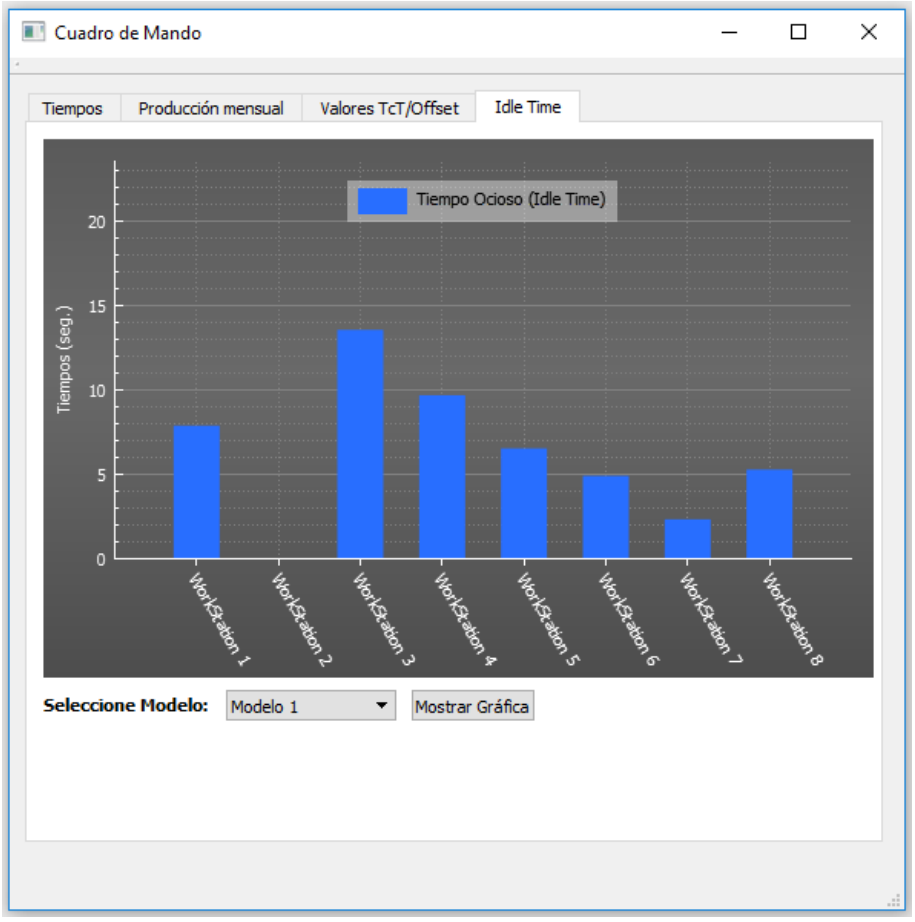


Figura 51: *Pestaña de Idle Time (Tiempo Ocioso).*

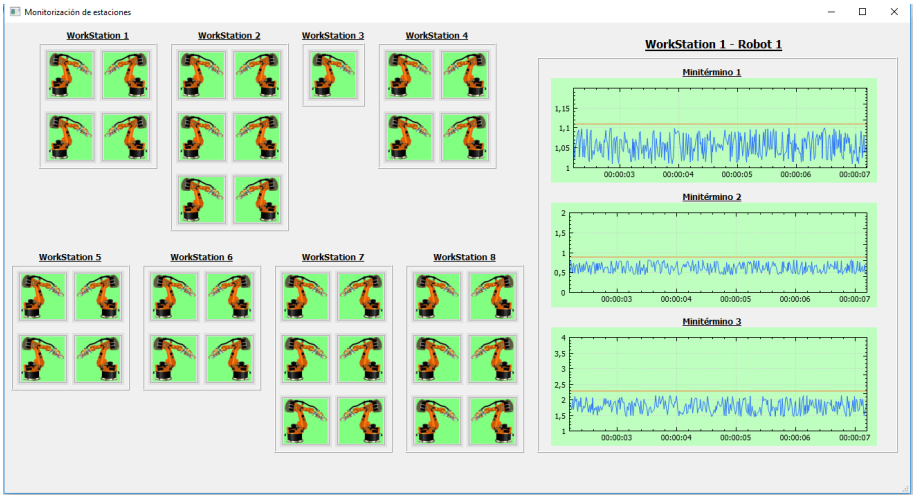


Figura 52: *Pantalla con el detalle de los tiempos de actividad de cada mini-término monitorizado.*

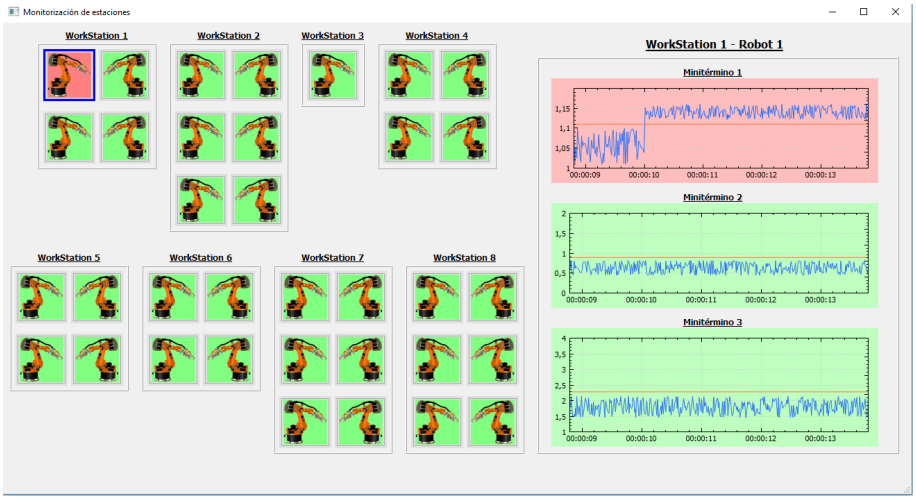


Figura 53: *Pantalla con la alerta de superación del umbral máximo.*

7. Conclusiones y trabajos futuros

El objetivo de este trabajo fin de grado es desarrollar una aplicación informática con un sencillo y a la vez completo entorno gráfico de usuario que permita fácilmente la detección de los fallos inminentes o a muy corto plazo en las estaciones de soldadura antes de que estos sean detectados por los sistemas de alarmas de la planta.

Esta aplicación supone en sí una anticipación a los actuales sistemas de detección de anomalías con el consiguiente ahorro económico, ya que mediante este sistema en tiempo real se pretende disponer de información suficiente del problema que se está produciendo mientras no se subsana el problema.

Las futuras correcciones a este trabajo podrán añadir mas funcionalidades que complementen las ya mostradas y que pueden llegar a planificar la perdida de beneficio económico, sumando nuevos KPI's a los ya programados.

Bibliografía

[1] http://www.fund-barredo.es/la-importancia-de-la-tecnologia-en-las-empresas/

[2] Accenture: Digital Business Era: Strech your boundaries. Accenture Technology Vision 2015

[3] Juan Carlos Latorre Iranzo. Dashboard para la toma de decisiones en la factoría Ford Valencia. Universidad CEU Cardenal Herrera. Julio 2017

[4] https://www.oracle.com/es/big-data/guide/what-is-big-data.html

[5] Dr. E. García Magraner. Análisis de los sub-tiempos de ciclo técnico para la mejora del rendimiento de las líneas de fabricación. Tesis Doctoral-Valencia (2016).

[6] Thorsten Pohlert. Non-Parametric Trend Tests and Change-Point Detection. July 30, 2018.

[7] Achim Zeileis, Friedrich Leisch, Kurt Hornik and Christian Kleiber. strucchange: An R Package for Testing for Structural Change in Linear Regression Models

[8] Karolos K. Korkas and Piotr Fryzlewicz. Multiple Change-point Detection for non-stationary Time Series Using Wild Binary Segmentation. London School of Economics. 2017

[9] Gordon J. Ross. Sequential and Batch Change Detection Using Parametric and Nonparametric Methods. July 28, 2015.

[10] Rafal Baranowski, Yining Chen, and Piotr Fryzlewicz, Narrowest-Over-Threshold Change-point Detection, Department of Statistics, London School of Economics and Political Science, Houghton Street, London, WC2A 2AE, UK. January 30, 2018.

[11] L.Li, Q.Chang, J.Ni. Data driven bottleneck detection of manufacturing systems. International Journal of production research ;47:18, pp 5019-5036 (2009).

[12] L.Li, Q.Chang, J.Ni. Real time production improvement through bottleneck control. International Journal of production research ;47:21, pp 6145-6158 (2009).

[13] https://es.wikipedia.org/wiki/Producci %C3 %B3n_en_cadena

[14] Boysen, N., Fliedner, M., Scholl, A., A classification of assembly line balancing problems. Jenaer SchriftenzurWirtschaftswissenschaft 12/06, University of Jena. 2006a.

[15] http://www.interempresas.net/MetalMecanica/Articulos/109390-La-simulacion-como-herramienta-de-valor-en-entornos-de-produccion-ajustada.html

[16] The Balanced ScoreCard: Translating Strategy into Action, Harvard Business School Press, Boston, 1996.

[17] https://es.wikipedia.org/wiki/Cuadro_de_mando_integral#Tipos_de_cuadro_de_mando

[18] http://www.cyfe.com/

[19] https://www.clicdata.com/bi/

[20] https://www.qt.io/ide/

[21] http://es.kantar.com/tech/m %C3 %B3vil/2017/marzo-2017-cuota-de-mercado-de-smartphones-en-espa %C3 %B1a-enero-2017/

[22] https://www.postgresql.org/

[23] https://www.pgadmin.org/